From Resemblance to Authority: Claim-Bounded Evidence for Android Software Lineage in AI-Mediated Supply Chains

HAOYI ZHANG, Xi'an Jiaotong-Liverpool University, China
HUAIJIN RAN, Nanyang Technological University, Singapore
XUNZHU TANG, University of Luxembourg, Luxembourg
KISUB KIM, Daegu Gyeongbuk Institute of Science and Technology, Republic of Korea
TEGAWENDÉ F. BISSYANDÉ*, University of Luxembourg, Luxembourg

Android lineage involves distinct claims about identity, composition, resemblance, derivation, process provenance, authorization, and behavior. This claim-centered review combines thematic searches, citation tracing, common eligibility criteria and seven-field coding to map 97 works, including 76 primary/system studies. Complete coding covers 85 works; the remaining records retain provisional interpretations. Our seven-level ladder separates resemblance from directional history, which similarity and chronology alone may leave ambiguous. Component matches depend on reference coverage and discriminating features; authenticated process records bind steps and products without proving benign behavior. AI-mediated development adds sibling-generation and context-recording ambiguities. An evaluation blueprint joins artifact, component, process, release-policy, and behavior evidence by APK digest or documented predecessor. For each claim, it identifies the observation, measurement, governing record, and plausible counter-history that an evaluation must distinguish. The resulting framework assigns explicit evidence obligations to each lineage claim.

CCS Concepts: • **General and reference** → **Surveys and overviews**.

Additional Key Words and Phrases: Android, software lineage, application repackaging, software supply chain, provenance, third-party libraries, code generation, evidence validity

1 Introduction

Two Android packages may look alike for many legitimate and illegitimate reasons. They may be separate builds of one code base, applications that embed the same advertising library, products generated from one template, regional variants signed by different subsidiaries, siblings produced from the same prompt, or an original and an unauthorized derivative. A detector can be accurate about a shared feature while still being unable to distinguish among those histories. The central problem is therefore not merely whether two artifacts are similar. It is whether the observation is strong enough for the relationship asserted by an analyst, marketplace, developer, or incident responder.

Parts of this distinction are already explicit in Android security surveys: Xu et al. separate legitimate privacy disclosure from malicious leakage and organize ecosystem roles, while Tam et al. distinguish analysis observations, detection outcomes and dataset limits [72, 83]. Repackaging studies examine code reuse and graph resemblance [13, 96], advertising and watermark evidence [21, 95], and malicious grafting [45]. Third-party-library (TPL) work examines common-library separation and detection [43, 46, 48]. Family and version identification must additionally handle shrinking, partial inclusion, and obfuscation [6, 87, 91]. Software-supply-chain work separates authenticated updates and delegation [35, 65] from recorded build steps and publication histories [51, 75]. Its questions concern actors, consumed

*Corresponding author: Tegawendé F. Bissyandé.

Authors' Contact Information: Haoyi Zhang, Xi'an Jiaotong-Liverpool University, Suzhou, China, hyeliozhang@gmail.com; Huaijin Ran, Nanyang Technological University, Singapore, Singapore, huaijin003@e.ntu.edu.sg; Xunzhu Tang, University of Luxembourg, Luxembourg, Luxembourg, realdanieltang@gmail.com; Kisub Kim, Daegu Gyeongbuk Institute of Science and Technology, Daegu, Republic of Korea, kisub.kim@dgist.ac.kr; Tegawendé F. Bissyandé, University of Luxembourg, Luxembourg, Luxembourg, tegawende.bissyande@uni.lu.

materials, produced artifacts, authorization and observer consistency. These bodies of work are complementary, but their guarantees do not compose automatically.

AI-mediated development makes the boundary more visible. A text-to-application system can generate template-based MIT App Inventor projects from natural-language descriptions, while Android development benchmarks evaluate generated projects and coding agents [27, 59]. Assistant studies examine insecure suggestions and user interaction [56, 57, 66], nonexistent package proposals [68], and sensitive-material reproduction [52]. The final APK alone does not record the causal history of iterative revisions. An application can therefore be novel at the byte level, similar at the behavioral or user-interface level, composed partly of inherited code, and produced through an authorized but weakly recorded process. No single binary relation captures all of those facts.

The practical consequence is a recurring category error. A high similarity score is reported as proof of copying; a component match is treated as proof of application ancestry; a valid signature is treated as proof of rightful authorization; an SBOM entry is treated as proof that the listed code is present and reachable; or a reproducible build is treated as proof that the inputs were approved. Each inference may become defensible with additional evidence, but the missing premise is often implicit. Once hidden, it is difficult to benchmark, transfer to a new workflow, or challenge during an appeal.

Prior synthesis is substantial: RePack reviewed 57 repackaging approaches and their comparison limits [42]; Wu et al. survey detection techniques and datasets [81]. Android TPL reviews cover module, family, version, maintenance and security questions [86, 89]. Supply-chain syntheses map attacks and design properties [29, 36, 55], complemented by research-direction and risk reviews [38, 79] and empirical SBOM studies [7, 69]. These predecessors supply the technical and methodological basis for the Android-focused claim composition developed here.

The question is claim-centered: *which observations support Android lineage and assurance propositions under explicit transformation and trust assumptions, and which bindings compose artifact, component, build, release, behavior, and AI-mediated evidence?* Distinct evidence objects and counter-histories motivate it; the incomplete 21-record matrix cannot establish a literature gap.

The survey answers four research questions:

RQ1: Claim structure. What distinct lineage and assurance claims recur in Android analysis, and which evidence objects can support each claim?

RQ2: Transformation robustness. How do rewriting, obfuscation, resource changes, library reuse, compilation, and generation alter the validity of artifact-level evidence?

RQ3: Cross-layer composition. How can artifact analysis, component evidence, authenticated process records, release authorization, and behavior analysis be combined without overstating any layer?

RQ4: AI-mediated development. Which assumptions become fragile when source and mobile artifacts are generated or revised through nondeterministic, tool-using systems?

Existing frameworks separate supply-chain artifacts, operations, actors, properties and defenses [29, 55]. AStRA's retrieved v2 identifies AI-generated software as future work, while Williams et al. directly discuss generation, testing and repair workflows [29, 79]. Building on those distinctions, this review offers an Android-focused seven-level claim ladder; a mechanism map linking objects, transformations, oracles, ceilings, and counter-histories; a digest-bound cross-layer assurance pattern; and an analysis of AI-mediated sibling generation, synthesized dependencies, hidden context, tool state, and model drift. The ladder adds propositions rather than ranking every higher level as universally better.

The review combines thematic keyword searches and citation tracing with common inclusion and exclusion criteria. Its 97-work map includes 76 primary/system studies; Section 2 details seven-field coding, reading coverage and the provenance of screening decisions.

The synthesis is that lineage evidence is *non-substitutable*: artifact and component evidence recover relationships when process records are absent; authenticated records establish direction and accountability; release controls bind distribution authority; and behavior analysis addresses security properties. Composition strengthens assurance only when artifacts, principals, times, policies, and transformations are explicitly bound.

Section 2 gives the review design. Sections 3–7 develop the claim model and domain syntheses; Sections 8 and 9 derive the cross-layer blueprint and agenda.

2 Review Design and Corpus

2.1 Review type, objective, and unit of analysis

This structured evidence synthesis combines thematic keyword searches, citation tracing, common eligibility criteria and seven-field coding to compare the claims and inference boundaries of a bounded literature corpus. Its analytic unit is a scholarly work contributing an Android artifact-comparison, component/version, provenance or reproducibility mechanism; an empirical supply-chain or AI-development study; or a structuring review or method source.

The coded unit relates evidence object, observation, transformation model, oracle or policy, claim ceiling, counter-history, and inference limit. Tool-level coding would hide stages such as module separation, signature construction, and version comparison; feature-level coding alone would also hide that clone pairs, family labels, repositories, signed policies, and rebuilds establish different propositions.

2.2 Search, snowballing, and corpus construction

Seeds included the Android repackaging benchmark review [42], the Android TPL review [89], broader Android surveys [70, 72, 83], and static-analysis/testing reviews [34, 44]. Backward and targeted forward snowballing followed supply-chain integrity, SBOMs, reproducible builds and AI-mediated development [9, 32, 80]. Wohlin's procedure requires inclusion before seed expansion and admits missed citation clusters; Kitchenham's 2004 report separates retrieval, quality assessment, extraction and synthesis. The recorded thematic searches covered Android repackaging, third-party libraries, software-supply-chain reviews and Android static analysis.

A 2024–2026 update added repackaging, TPL and adjacent review records [61, 81, 86]. Research-direction and supply-chain reviews [23, 38, 79] and empirical SBOM studies [7, 69] were tracked separately. A candidate SBOM review was excluded after its official record reported withdrawal and no paper was available for coding. The screening ledger contains 105 decisions: 97 inclusions, seven scope exclusions and one withdrawn publication. Source-location and retrospective discovery rows accompany retained works; the initial database-hit denominator is unavailable.

The map contains 97 works: 76 primary/system and 21 review/method records. 74 primary/system and 93 total records are venue-published; the remainder are three preprints and one method report. Legacy depth labels (61 full-or-selected-source; 36 abstract-or-selected-passage) do not certify passages read. The locator ledger records PDF page/section or numbered primary-HTML locations for 88 works and lacks such checks for 9 legacy records; neither count measures complete semantic coding.

Table 1. Corpus strata and their role. Counts describe the retained evidence map, not field prevalence.

Stratum	Works	Main evidence objects	Principal use
Android artifact analysis	34	DEX code, graphs, resources, UI, behavior, signatures, clone pairs	Bound artifact-level identity, resemblance, and directional claims
Third-party libraries	17	Module boundaries, signatures, versions, update histories	Separate composition, version, reachability, and application ancestry
Supply-chain integrity	17	Metadata, attestations, logs, roles, builds, CI workflows	Establish principals, steps, authorization, transparency, and reproducibility
AI-mediated development	8	Prompts, generated source, dependencies, interactions, accepted output	Identify generated transformations, missing context, and causal-record limits
Reviews and method	21	Taxonomies, protocols, benchmark critiques, design properties	Adversarially position scope and normalize evaluation obligations

The heterogeneous corpus is not pooled into a detector ranking: candidate generation, transformations, labels, periods, and counting units differ, and benchmark construction can dominate apparent results [8, 28, 42]. The synthesis therefore compares mechanisms and inference validity.

2.3 Corpus records

The evidence package separates the representation consumed for coding from the canonical bibliographic record. Each retained work has a corpus row, BibTeX entry, source-location boundary, retrospective discovery row, and publication-status record. DOI-bearing entries use the DOI as canonical identifier; other records use an official venue, archive, or report locator. Automated checks require title and year agreement, unique identifiers, and consistency between venue/preprint status and entry type. A withdrawn SBOM candidate is preserved only in an exclusion ledger and historical regression fixture. These checks establish record-level consistency, not a comprehensive retraction-registry search or independent confirmation of every interpretation.

2.4 Prior-review adversary matrix

The 21 review/method records are compared across Android repackaging, component/version evidence, provenance, authorization, behavior, AI mediation, and an explicit claim ceiling with cross-layer binding. Table 2 uses 1 for located direct treatment, P for located partial treatment, 0 for absence supported by complete scoped reading, and ? for unresolved. Located readings support 75 direct/partial cells and 41 scoped absences; 31 cells remain unresolved. Eighteen records have content checks; three lack a current substantive check. The matrix retains unresolved judgments separately from complete-scope absences.

RePack's common-predecessor example supplies partial ceiling treatment [42]. The TPL review treats component privilege policies and stage-specific limits in the retrieved author version [89]. Wohlin bounds seed-cluster discovery and replication inferences; Kitchenham distinguishes unknown conduct, duplicate reports and incomparable outcomes [32, 80]. These are partial method-level ceilings, not integrated artifact/principal/policy bindings. Their matrix decisions reuse the existing complete-PDF assessments. AStRA's retrieved 2024 v2 identifies AI-generated software as future work; Williams et al. directly discuss LLM supply chains [29, 79]. Xu et al.'s update-actor flow supplies partial lifecycle context [83]. These distinctions determine the corresponding matrix codes.

Direct all-dimension coverage is undetermined: unresolved rows could supply a witness. The codes position the retrieved reviews rather than establish a literature-wide gap.

Table 2. Selected prior-review matrix.

The columns denote Android repackaging (A), component/version evidence (C), process provenance (P), authorization (U), behavior (B), AI-mediated generation (G), and claim-ceiling/binding rules (X).

Prior synthesis	A	C	P	U	B	G	X
[42]	1	P	0	0	P	0	P
[89]	P	1	0	P	P	0	P
[29]	0	P	1	1	1	P	1
[55]	0	P	1	1	P	0	P
[81]	1	1	?	?	P	?	P
[86]	P	1	P	P	P	0	P
[61]	?	?	?	?	?	?	?
[79]	0	1	1	1	P	1	P
[38]	0	P	P	P	P	P	0
[23]	0	P	P	P	P	0	P

Version and scope also govern the newer review rows. Latsiou and Lambrinoudakis cover supplier governance, standards, CI/CD integrity and partial generative-AI risk, not an evaluated generated-code history [38]. Gokkaya et al.'s retrieved May 2023 v1 already discusses delegated authority, in-toto, permissions, patching and presence-versus-exploitability limits [23]. Its incident charts use 106 selected records from 2018–2021; this is not a current incidence estimate. The differently titled 2026 final paper has not been compared with those retrieved bytes. Scoped matrix codes therefore refer to the inspected versions, not every later revision of a cited work.

2.5 Inclusion, exclusion, and source-depth rules

Works were included when their technical object could support an Android lineage or release claim. Clone/malware systems supplied reusable artifact observations; supply-chain systems supplied source, build, repository, or release evidence; and AI papers generated mobile software or measured security-relevant generation workflows. General model, GUI-agent, or prompt-injection work without a software-artifact lineage object was excluded.

Publication status and evidential role were separate. Preprints expose emerging workflows and method reports guide procedure, while venue status identifies peer-reviewed primary evidence. Every record has a scholarly/official locator and version-status row; known withdrawn or retracted candidates are quarantined. Records without acquired PDFs include an Android review and a watermark study [62, 70], repackaging and library studies [19, 73, 84], a library-detection study [82], and supply-chain detection/signing records [50, 54]. Reichert et al. also remain outside the acquired set [61]. Gokkaya et al.'s retrieved v1 has complete scoped coding, with comparison to the differently titled final text still unresolved [23]. Bibliographic availability and technical reading are tracked separately.

2.6 Claim-level extraction and recheck

The seven extraction fields are evidence object, observation, transformation model, oracle or policy, claim ceiling, counter-history, and inference limit. Field population and source location are not semantic reading. Broad categories such as "renaming or library reuse" do not mean every cited study tested each operator; counter-histories and inference limits are our classifications. Source-specific interpretations require the study's actual object, transformation, oracle, and version.

Table 3. Evidence-strength notation.

Grade	Evaluation basis	Typical strength	Typical limitation
E0	Illustrative example	Makes a mechanism concrete	No discriminating evaluation
E1	Curated positive cases	Demonstrates feasibility	Does not estimate false positives or open-world behavior
E2	Positive and negative labeled corpus	Supports discrimination under a stated oracle	Labels may encode the method's assumptions
E3	Transformation-aware or adversarial corpus	Tests robustness to explicit changes	Covers a finite transformation family
E4	Temporal, cross-market, or independently constructed validation	Reduces leakage and construction bias	Transfer beyond sampled ecosystems remains uncertain
E5	Authenticated process evidence or independent reproduction	Supports a stated provenance or equality proposition	Does not imply semantic safety or policy authorization

For example, FlowDroid analyzes lifecycle-aware taint paths, not renaming robustness [5]; LibScout depends on package hierarchy and version-discriminating interfaces [6]; ATVHunter distinguishes library from version identification [87]. Legacy category fields cannot establish per-operator robustness.

The cumulative ledger contains 90 dated same-process locator rechecks (92.8%). A separate reader-assessment ledger records complete retrieved-PDF/seven-field coding for 85 of 97 works, covering every acquired PDF. Each assessment relates the study's observation, transformation and oracle to the statements used in this synthesis. The remaining 12 records have no acquired PDF through the attempted routes, although some have selected HTML readings.

Supplementary tables contain 85 located records and 595/679 locator-to-field bindings. Counts are generated from these exact record sets, with source reports separated from analytical classifications. Complete reading applies to the retrieved body, references and included appendices. Version boundaries remain explicit: RepDetector's author copy differs from its earlier selected HTML representation; MaMaDroid's 2019 extension differs from the conference paper; and AppForge v1 has 21 physical pages. Other author/preprint copies retain their stated final-publication distinctions.

Coding provenance. The same extraction workflow supplies these assessments and selected-passage cross-checks, without independent second-human coding. Software checks bindings and calculations; source interpretation is recorded in the reading assessments.

2.7 Evidence-strength notation

The claim ladder states *which proposition* is supported; an orthogonal grade states *how it was evaluated*.

The two axes must remain separate. A valid signature may strongly bind a key and artifact while saying nothing about functional equivalence. A transformation-aware benchmark can strongly evaluate resemblance robustness without identifying the authorized publisher.

2.8 Synthesis and contradiction procedure

Synthesis first grouped observation families, then assigned each a ceiling and counter-history, and finally mapped the bindings needed across the Android lifecycle. Contradictions denote incompatible objects, denominators, times, policies, or oracles rather than numerical disagreement alone. This is not vote counting: repeated observations cannot convert

resemblance into direction or provenance into benign behavior. Seven synthesis propositions retain named support, counter-histories, and boundaries in the claim ledger.

2.9 Deterministic traceability and negative controls

The artifact tests cross-ledger identities, publication status, source boundaries, dated recoding transitions and generated manuscript totals. Mutation controls exercise invalid changes to keys, titles, dates, fields and excluded-record status. Complete reader assessments are tracked separately from these structural checks.

The retained historical pilot exercises unknown handling against an independently expressed completion rule. Its exhaustive 3×3 test covers 19,683 ternary matrices and 262,144 binary completions. This validates bookkeeping semantics; literature interpretation remains a reading judgment. Counts and display rows are regenerated from current ledgers before compilation.

2.10 Validity limits

The review is a bounded evidence map with complete seven-field coding for 85 records and unresolved interpretations for the remainder. Its search denominator and independent second coding are unavailable. Detector accuracy and prevalence are not pooled across incompatible study designs; unresolved matrix cells keep literature-wide absence undetermined.

3 An Evidence Model for Mobile Software Lineage

3.1 Why lineage is not one relation

The term *lineage* is used for at least four different relations. A structural relation states that two artifacts share material. A historical relation states that one artifact was derived from another or that both descend from a common source. A procedural relation states that an artifact was produced by a recorded sequence of steps and principals. A normative relation states that the production or release was authorized. Security and policy questions add a fifth relation concerning behavior. These relations can coincide, but none entails all the others.

Let an evidence item be represented as

$$e = (o, \phi, r, m, q, a),$$

where o is the observed object, ϕ is the feature or record extracted from it, r is the relation being inferred, m is the transformation model, q is the oracle or policy against which the inference is evaluated, and a is the set of assumptions. The tuple makes the observations and inference premises of each mechanism explicit.

For example, a fuzzy code signature may compare methods after identifier normalization. Here, o is DEX method code, ϕ is a normalized signature, m includes identifier renaming but may exclude control-flow restructuring, and q may be a corpus of known repackaged pairs. The supported relation is resemblance under m. Direction is not part of that observation unless the oracle or auxiliary evidence supplies it. By contrast, an in-toto link observes authenticated step metadata and binds materials to products under a layout [75]. It supports a process relation under key, layout, and metadata assumptions, but does not establish that two programs are semantically equivalent.

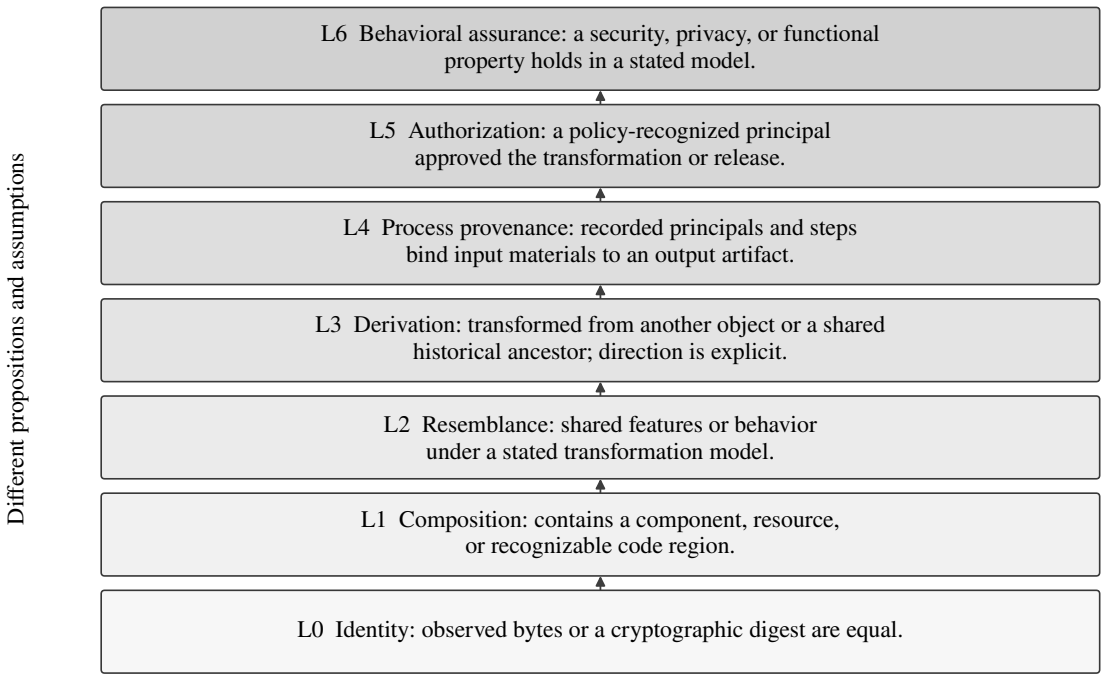


Fig. 1. Android lineage claim categories. Arrows indicate presentation order, not logical implication.

3.2 The seven-level claim ladder

Figure 1 summarizes seven numbered claim categories. They are neither a maturity model nor an implication chain: a behavior claim can be supported without reconstructing provenance, and authorization does not establish resemblance. The order introduces different propositions, not cumulative guarantees.

L0*: Identity.* Cryptographic hashes, package digests, and reproducible-build comparisons can establish equality of the observed bytes with negligible collision probability under the selected hash assumptions. Identity is useful for artifact distribution and cache verification. It does not reveal why the bytes are equal, who authorized them, or whether they are safe. Two parties can distribute the same malicious binary.

L1*: Composition.* Component detectors, software bills of materials, package manifests, and static analyses address whether an artifact contains a component or a recognizable region. The TPL review organizes composition difficulties [89]; detector-specific premises cover changed package structure, dead-code removal and partial embeddings [6, 77, 91]. Composition also has granularity: identifying a library family is weaker than identifying an exact version, and identifying bytes is weaker than showing that vulnerable code is reachable.

L2*: Resemblance.* Most clone and repackaging detectors occupy this level. They establish that code, graphs, resources, UI, or behavior are similar under a threshold and transformation model. DroidMOSS, Juxtapp and AnDarwin illustrate fingerprint and code-reuse choices [14, 26, 96]; WuKong and SimiDroid add filtering and multi-level comparisons [41, 76].

Resemblance can be highly probative, particularly when independent feature families agree, but it does not by itself provide direction.

L3*: Derivation.* A derivation claim states that B descends from A, that A descends from B, or that both descend from a specified ancestor. Authenticated input–output records or controlled benchmark construction can establish direction under their trust assumptions. Timestamps, market chronology, ownership, and deployed marks can support adjudication, but order or ownership alone does not exclude a shared predecessor. Repackaging papers often use known pairs or market metadata to operationalize derivation; the feature detector and pair-construction oracle should be reported separately [28, 42].

L4*: Process provenance.* Process provenance binds materials, commands, environments, and principals to products. in-toto links and layouts are a direct example [75]; CI workflow studies show why the integrity of the recording and execution environment also matters [33]. Provenance can make direction explicit because it records a step from inputs to output. It remains conditional on key control, policy, collection completeness, and the absence of unrecorded paths.

L5*: Authorization.* Authorization adds a policy judgment: the principal was allowed to perform or approve the step or release. TUF's delegated roles and Diplomat's repository delegations exemplify policy-scoped authority [35, 65]. An Android APK signature establishes possession of a signing key for that package artifact; whether the key holder is the rightful publisher is an external identity and policy question. Authorization is therefore not reducible to the presence of any valid signature.

L6*: Behavioral assurance.* Security, privacy, and functionality concern what the artifact does. Static taint analysis, malware classification, policy analysis, dynamic testing, and formal verification support claims under their models [4, 5, 49]. A layout can require such inspections [75]; the support then comes from that property-specific check, not metadata authentication alone. Conversely, a behavior detector can find malicious functionality without reconstructing the artifact's full history.

3.3 A worked claim-adjudication example

Consider two Android packages, A and B, collected from different channels. Their digests differ; both contain the same advertising-library family; normalized first-party call graphs, layouts, and images correspond after localization. Package A also has an attested build from a named source revision and a store release under a delegated publisher role. Package B uses another key and lacks an authenticated production record. Dynamic analysis reports that B contacts an additional endpoint. This synthetic example illustrates adjudication rather than reporting a measurement.

The observations license separate statements. Different digests reject L0 identity. The shared library supports family-level L1 composition while leaving version, modification, and reachability unresolved. Agreement across first-party code and resources supports L2 resemblance under the detector's transformation model, but does not select a direction: a common template, common predecessor, authorized fork, and copying either way remain compatible histories. The authenticated record supports L4 only for the materials, steps, principals, and product digest it names; delegation can add scoped L5 authorization for A. Neither claim transfers to B, and a different valid key is not by itself proof of unauthorized release. The endpoint finding is a L6 observation about the analyzed build and environment, not a derivation oracle.

Later evidence should revise only the subclaim it resolves. An authenticated parent and ordered transformation could establish directional L3 or process-level L4; a shared authenticated predecessor would instead establish sibling

Table 4. Typical claim ceilings and missing links. The last column names the evidence normally required for a stronger inference.

Evidence object	Ceiling	What it directly supports	Evidence needed for a stronger claim
Exact digest or byte comparison	L0	Identity of observed bytes	Authenticated origin, policy, or behavior analysis
Component signature or SBOM entry	L1	Component presence or asserted composition	Exact version, reachability, and authenticated generation
Code, graph, resource, UI, or trace similarity	L2	Resemblance under a feature and transformation model	Directional oracle, chronology, ownership, or process record
Known original–derivative pair	L3	Derivation for the constructed pair	Generalization to open-world candidates and authorization
Authenticated step attestation	L4	Recorded input–step–output relation and principal	Authorization policy, complete capture, and trusted execution
Delegated signature or release approval	L5	Policy-scoped authorization of metadata or artifact	Semantic equivalence, vulnerability, privacy, and behavior evidence
Static, dynamic, or formal behavior evidence	L6	Property within the analysis or test model	Broader environment coverage and provenance of analyzed inputs

derivation. The correct report therefore preserves competing histories, binds every result to its exact object, and keeps retrieval, historical adjudication, authorization, and behavior distinct even when a store policy combines them into one action.

3.4 Claim ceilings of common evidence objects

Table 4 records typical claim ceilings. "Typical" matters: an implementation can combine additional evidence and move beyond the base ceiling. The purpose is to prevent that additional evidence from disappearing inside the name of the base mechanism.

3.5 Counter-histories as a validity test

A useful way to evaluate an inference is to construct a counter-history: a plausible history that produces the observation but falsifies the claimed relation. Four counter-histories recur.

Common-predecessor ambiguity occurs when two applications inherit code from an earlier version or template. Pairwise similarity cannot establish which is the source. *Component contamination* occurs when a large shared library dominates the feature space and makes otherwise unrelated apps appear close. *authorized divergence* occurs when a legitimate maintainer refactors, obfuscates, localizes, or regenerates an application so heavily that its authorized successor looks dissimilar. *authenticated malice* occurs when a compromised or maliciously authorized pipeline produces a correctly signed artifact. These histories respectively challenge direction, attribution, recall, and security inferences.

Counter-histories do not prove a detector useless. They identify the assumptions under which its output is probative. A marketplace may reasonably combine similarity with publisher identity, upload time, signing history, and user reports. The resulting decision is a fusion system; it should not be described as if the similarity feature alone established plagiarism.

3.6 Evidence fusion and non-substitutability

Evidence fusion is useful when observations challenge different counter-histories under explicit joint assumptions. Code and UI resemblance can weaken a shared-library-only explanation; an authenticated build record can supply

direction; transparency can expose inconsistent releases; and behavior analysis can test the exact output. These roles do not require statistical independence. If probabilistic scores are combined, however, dependencies among features and shared label errors must be modeled rather than counting correlated observations as independent corroboration.

For an artifact A, an assurance case can be expressed as a set of linked claims:

$$C(A) = \{C_{\text{comp}}, C_{\text{rel}}, C_{\text{prov}}, C_{\text{auth}}, C_{\text{beh}}\}.$$

The links are explicit bindings: a digest joins an attestation to an APK; a source revision joins a repository event to a build; a library version joins component detection to a vulnerability record; and a package signature joins an artifact to a release identity. If any binding is missing, the layers can refer to different objects while appearing to support one conclusion.

This model frames the remaining sections. Android repackaging research is rich in L1–L3 evidence. Supply-chain systems are rich in L4–L5 evidence. Android program analysis contributes to L6. AI-mediated development changes the transformations and principals that must be represented across all levels.

4 Android Repackaging and Artifact-Level Evidence

Android repackaging research began from a concrete abuse pattern: obtain an application, modify it, re-sign it, and redistribute the result. The modification may add advertising, remove payment checks, graft malicious code, change branding, or camouflage an application in a market. This history makes repackaging a natural lineage problem, but it also creates a temptation to collapse several tasks into one label. Candidate retrieval, similarity measurement, pair classification, direction assignment, maliciousness assessment, and authorization judgment are separate stages.

The review by Li et al. documents 57 approaches and emphasizes the difficulty of reproducing and comparing their results [42]. That finding remains a useful organizing constraint. A survey of evidence should not ask only which feature a detector uses. It should ask what unit is compared, which transformations are modeled, how pairs are labeled, whether common libraries are removed, whether direction is part of the label, and whether the evaluation contains realistic negatives. Those choices determine the claim ceiling.

4.1 Code and structural resemblance

Early scalable systems represented applications through fingerprints, normalized code, or structural abstractions. DroidMOSS uses fuzzy hashing to detect repackaged applications in third-party markets [96]. DNADroid, introduced in Attack of the Clones, retrieves metadata candidates and then compares method data-dependence graphs; AnDarwin instead uses successive approximate clusterings [13, 14]. Juxtapp compares code-feature sets and follows up selected cases, whereas WuKong filters candidates before detailed comparison [26, 76]. Candidate selection and confirmation thus have system-specific representations and costs. Earlier studies of application plagiarism and market-level reuse also show that the same observed reuse can correspond to copying, legitimate components, or product-family evolution [58, 63].

The reuse study uses declaration-only class signatures; global reuse can be the union of many apps, while mutual containment yields legitimate same-developer and framework groups [63]. Its reuse-prone category sample is not market-random. Signature equality therefore does not establish unauthorized copying.

Normalization trades transformation resilience against collisions. Ignoring names helps with renaming; graphs retain structure but change under inlining, reflection or packing. CodeMatch compares normalized structures, whereas DAPASA classifies sensitive static call subgraphs under historical labels, not a pairwise history oracle [18, 22]. SimiDroid already combines method, component and resource profiles, but its symmetric score and pair-order change explanation

do not establish direction; library exclusion can raise or lower similarity [41]. RepDetector compares selected symbolic outputs with bounded loops and omitted native calls, not whole-program equivalence [25].

CLANdroid retrieves functionally related apps, not labeled repackages [47]. Its sensor variant improves recommendation ranks yet worsens participant ratings; empty-profile ties and filtering-dependent rank reversals show why candidate set and oracle must accompany similarity results.

Robustness to renaming can recover derivatives and also increase common-framework matches. The representation must fit the asserted relation and candidate population.

Several works explicitly pursue scale. Achieving Accuracy and Scalability Simultaneously, WuKong and MassVet use staged processing or compact features at scale [11, 12, 76]; APPraiser separately studies large-market relatedness categories [30]. Scale changes the evidence problem. In a small curated pair set, every candidate may be known to have a related counterpart. In a market-scale setting, most pairs are negatives, labels are sparse, and a small false-positive rate can generate an unmanageable review queue. Reporting accuracy on balanced pairs is not enough to estimate this operational burden.

Incremental matching is already measured, though under different contracts. The centroid system includes disassembly and matching for selected new apps; AnDarwin times an online full-app path while running library and partial-app work periodically [11, 14]. WuKong's per-submission time is proposed rather than measured [76]. Juxtapp measures new–new and new–old updates, but its 7,000-APK update includes both 57 minutes of distribution and 104 minutes 48 seconds of completion, and its compact feature vectors require recovery tables and intermediates [26]. Bootstrap also matters: AnDarwin's clustering follows 4.47 days of extraction, the centroid comparison follows about forty hours of generation, and WuKong separates disassembly, 28 hours of subpackage feature work and later matching [11, 14, 76]. These source-reported stages and hardware do not yield a same-contract speed or memory ranking; Juxtapp's retrieved author manuscript is not its final representation.

4.2 Code heterogeneity and piggybacking

Piggybacking narrows the lineage question by looking for a largely reused host with injected or changed code. The systematic study of malicious code grafting documents how code is added to existing applications [45]. DR-Droid partitions a recovered class-dependence graph and classifies its regions without requiring a reference original [74]. Its operational partition ignores edge direction, whereas Definition 3 requires directed reachability between ordered vertex pairs; the mapping proof does not resolve this distinction or prove independence in the complete Android runtime. Integrated benign/malicious regions and omitted static dependencies remain outside its intended separation model. DAPASA instead classifies one selected sensitive subgraph under empirical invocation/cohesion premises [18]. These classifiers can motivate further investigation of anomalous regions, but neither supplies an insertion history. Applications commonly combine first-party code, generated code, libraries, and modules produced by different teams.

A stronger piggybacking claim therefore needs at least two bindings: a relationship between the candidate and a plausible host, and evidence that the differing region corresponds to an added transformation rather than ordinary variation. Temporal market records, signing changes, known original–piggybacked pairs, or authenticated source histories can supply direction. Behavior analysis can then assess the inserted code. Without those bindings, a detector may accurately locate anomalous code while remaining uncertain about origin and intent.

This distinction also explains why malware classifiers are relevant but not interchangeable with lineage detectors. DREBIN and MaMaDroid infer maliciousness from application features and behavioral abstractions [4, 49]; FlowDroid supports precise information-flow analysis within an Android lifecycle model [5]. Their observations can strengthen L6

claims about a candidate. They do not establish that the candidate was copied from a specific application. Conversely, a near-perfect clone match does not establish that the changed or unchanged behavior is malicious.

DREBIN explains static feature-score contributions, not runtime events [4]. Its scanner-derived labels partly overlap its comparator; legitimate SMS yields false positives and a downloader family is difficult. The measured on-device model and analysis time exclude total memory, training and maintenance.

MaMaDroid's 2019 extension evaluates class abstraction and a frequency variant, FAM, distinct from the conference paper [49] (https://arxiv.org/abs/1711.07477v2). Static Markov features are not executed traces. FAM wins some old-benign-data comparisons, while finer class abstraction gives little same-year gain and worse older-to-newer performance. Different extraction, samples, filtering and FAM no-result rules preclude a controlled sequencing-versus-frequency claim. Millisecond classification omits extraction; memory allocation is not peak use, and class clustering uses all datasets rather than demonstrated training-only fitting.

4.3 Resources, user interfaces, and externally visible behavior

Resources and UI can remain recognizable after code rewriting [20, 67, 90]; DroidEagle also uses layout evidence [71]. These signals help detect presentation similarity and brand impersonation, but templates and shared assets are legitimate alternatives. Localization, restyling and sampled UI states bound both recall and interpretation: a match is not source derivation or full functional equivalence.

FSquaDRA's largest two clusters were manually judged legitimate because shared ActionBarSherlock files dominated; AndroGuard also scored them highly [94]. Agreement did not establish copying. Its full-package hashes include code and manifests, not just visual resources. Reported pair-comparison throughput excludes 15.10 hours of loading and quadratic all-pairs work.

ResDroid fuses core layouts and event signatures; ViewDroid uses activity-navigation graphs [67, 90]. Their template oracles differ: the latter accepts converter-generated apps as positives, the former reports shared builders as false alerts. Juxtapp also distinguishes common-builder and same-author reuse [26]; ResDroid's key-variant cases show why neither template reuse nor certificate mismatch fixes ownership.

DroidEagle compares layout trees or skeleton hashes with trusted references and then certificate fingerprints [71]. It distinguishes certificate presence from authenticity, yet a signer mismatch is still not a scoped authorization verdict. Its bounded reimplemented controls include a layout-hash miss; the 0.8-second result measures selected hash generation, not full verification. Printed corpus and cloud-source totals disagree, precluding a reconciled prevalence claim.

Distinctive first-party code plus layout weakens a shared-library-only explanation, but generated bindings can correlate the features; fusion must report their underlying artifacts.

4.4 Watermarks, deterrence, and owner-origin evidence

AppInk takes a different approach: a private companion manifest app drives recovery of a developer-chosen runtime graph watermark [95]. Its five-application robustness experiment evaluates distortion transformations; removal and addition are analyzed but not tested. The source authors repaired an initial ADAM renaming failure before reporting successful recognition, not an independently reproduced native result here. The three-app timing study reports 7–28 seconds for recognition versus 1.3–1.6 seconds of incremental extraction, without a complete build, embedding, memory or CPU account. A verified mark can supply designed-origin evidence when prior deployment and claimant identity are bound. AppInk's trusted-arbitrator/confidential-manifest premise and first-submission argument matter: the paper explicitly admits that first publication in another market escapes its chronology argument.

A mark needs prior identity binding, specificity, and resistance to removal and forgery. A contractor or licensee may legitimately hold marked source: origin (L3 with deployment history) is not authorization (L5 under policy). Deterrence is also distinct from detection; evaluation should cover survival, false claims, collusion, cost, and changed detector features.

4.5 Transformation models and adversarial validity

Android applications undergo ordinary and adversarial transformations: compiler changes, shrinking, optimization, identifier renaming, control-flow rewriting, string encryption, packing, library updates, resource replacement, and signing. DroidChameleon's historical stress test applied selected transformations to six old samples and ten anti-malware products chosen to detect the originals [60]. Its preservation contract concerns selected security-relevant behavior, not every observable event; function outlining/inlining appears in the design but not the implemented experiment. The lesson transfers to lineage analysis: a detector's transformation model is part of its claim, not an implementation detail. Finite transformations and detection changes do not by themselves establish present-day failure rates or complete runtime equivalence.

Transformation robustness should be described with a matrix rather than a binary "obfuscation resistant" label. A feature may survive renaming but fail under dead-code removal; survive resource edits but fail under package flattening; or tolerate local rewriting while failing when an application is regenerated from a high-level specification. CodeMatch and SimiDroid supply code and multi-level views [22, 41]; FSquaDRA and the resource-based evaluation supply resource views [20, 94]. These occupy different cells in the matrix. For the representations and artifact domains reviewed here, robustness is demonstrated only for the stated operator families; coarser abstractions can also match distinct artifacts. These results do not establish an impossibility for every representation and transformation model.

There is also a direction asymmetry. A detector optimized for finding transformed copies of a known original has a reference artifact and can compute expensive targeted features. Open-world clone discovery must compare many artifacts without knowing the ancestor. Evaluations should state which setting is measured. Results from reference-based detection do not automatically transfer to all-pairs market discovery.

4.6 Ground truth, datasets, and comparison validity

Repackaging ground truth can come from controlled changes, developer records, chronology, source relationships, deployed marks or manual investigation. Controlled pairs know direction but may lack field diversity; market pairs add realism with uncertain history. Malware-family and certificate labels are not derivation oracles.

RePack uses similar code, different certificates and DEX-time ordering, but warns about manipulated timestamps, multiple generations and siblings of a common predecessor [42]. Its three-tool comparison and separate SimiDroid result use different completed-pair denominators; neither measures open-world false positives. AndroZoo broadens artifact access but does not supply authenticated histories [3]. Benchmarks need pair formation, direction evidence, transformations, library handling, failures and hard negatives.

DNADroid makes direction a non-goal and admits a common predecessor [13]; DroidMOSS assumes an official-market original [96]. AdRob cautions that certificates do not identify owners [21]. WuKong leaves direction unresolved and discusses the unsoundness of available heuristics [76]; APPraiser's market/download rule depends on the original being present [30]. A nominal host may already be repackaged [45]; DroidKin relatives can share public debug certificates without one author [24]. The directional evidence must therefore accompany each pair label.

Table 5. Representative Android artifact-evidence families. The claim ceiling assumes no auxiliary provenance or policy evidence.

Family and examples	Observation and modeled resilience	Ceiling	Recurring failure boundary
Fuzzy and normalized code [22, 96]	Method or application fingerprints; often tolerates renaming and local edits	L2	Common libraries, templates, coarse collisions, unseen rewrites
Graph and structural abstraction [13, 14]	Method data-dependence graphs or approximate graph-derived features	L2; L3 with external pair oracle	Graph rewriting, incomplete recovery, direction supplied by labels
Market-scale matching [11, 12, 76]	Indexed features and system-specific filtering; MassVet also raises conditional risk alerts	L2; L6 in MassVet	Base-rate effects, sparse labels, candidate-stage false negatives
Resource sets [20, 67]; UI views [71, 90]	Resources, layouts, images, static navigation graphs, or rendered views	L2	Templates, localization, state coverage, deliberate restyling
Embedded marks [95]	Designed watermark verified in a candidate	L3 with deployment record	Removal, forgery, licensed reuse, weak identity binding
Malware classification [4, 49, 74]; flow/subgraph evidence [5, 18]	Static features, sensitive subgraphs, flows, anomalous regions	L6 for modeled property	Does not identify ancestor; analysis unsoundness or incomplete environment

The piggybacking study deliberately constructs obvious pairs: different certificates, same-version/SDK metadata, an earlier nominal creation time, at least 80% code similarity and added code yield 950 pairs in the retrieved author text [45]. Same-developer and resource-only changes are excluded. Its resource and library-region findings challenge blanket preprocessing assumptions, but its static flows are modeled paths, not observed leak events; scanner labels and DEX timestamps are not authenticated histories. The paper explicitly retains uncertainty about the true first original. These qualifications are part of its evidence, not a gap supplied by this survey.

The earlier plagiarism study uses a different oracle [58]. In its 7,600-app repository, 500 benign test uploads are removed before query, while 500 synthetically changed uploads retain their parents. Shared library method fingerprints can match the wrong app. Its million-entry scalability sets resample those 7,600 binaries, rather than supplying millions of independent observations. Finite near-copy accuracy, repository-presence assumptions and resampled index cost therefore remain separate claims.

Quantitative results inherit their oracles. DNADroid's zero false-positive report checks returned pairs without a labeled negative population or ownership oracle [13]. CodeMatch's description-selected labels and reimplemented controls do not turn its historical 15.7% app classification into copying prevalence [22]; DroidMOSS's selected-positive misses and candidate errors have different denominators [96]. AdRob's 14% impression-loss estimate assumes owner merging, original-reference presence, and user substitution; it is neither measured dollar loss nor an unconditional lower bound [21]. AnDarwin's DNADroid-based sampled-cluster validation leaves market-wide misses and graph collisions unresolved [14]; the centroid system's selected returned groups and WuKong's sampled cases likewise cannot measure population error [11, 76]. FSquaDRA's certificate-stratified correlation and threshold are not authenticated history labels [94].

The same distinction applies beyond clone pairs. DAPASA's reported 95%/0.7% rates have unspecified fold-specific fitting and conflicting benign denominators; its preprocessing is slower than its permission/API control despite fewer

features [18]. DR-Droid's printed miss rate and timing units conflict with its counts/prose [74]. APPraiser retains an unknown category and confirms a sample of returned clones, not whole-market recall [30]. SimiDroid drops extraction failures and finds no significant method-score advantage over AndroGuard at its stated threshold [41]. The balanced resource evaluation excludes unprocessable apps, lacks a specified nested feature-selection split, and optimizes a control threshold on the full dataset [20]. These source-reported numbers cannot be pooled or repaired into one accuracy ranking.

Operational cost has the same boundary problem. MassVet's online path includes preprocessing, but bootstrap databases and memory are separate; its suspicious-app total includes unconfirmed cases and unreconciled category counts [12]. ResDroid's inspected returned groups and selected known-positive misses use different denominators, while ViewDroid caches extraction and graph construction before pair matching [67, 90]. Huang et al.'s transformation framework admits non-runnable outputs and conversion failures, so successful static conversion is not an independent behavioral oracle [28]; ViewDroid has unfinished compositions and ResDroid fails under class encryption [67, 90]. AndroZoo supplies snapshot digests, not complete market history or validated scanner labels: in its historical sample, changing the flag threshold from one to ten engines changes the flagged share from 22% to 1% [3]. F-Droid variants may legitimately remove tracking, and collection access does not imply redistribution rights. No original detector or transformation was executed for this review.

A useful negative set is not a random sample of unrelated applications. Random negatives are often easy and can overstate discrimination. Hard negatives include applications from the same developer, versions of one legitimate product, applications built from the same template, apps sharing large libraries, and independently implemented apps with similar UI or functionality. These cases test whether the detector confuses legitimate relatedness with unauthorized derivation.

Temporal leakage is another risk. If a detector's signature database, threshold, or feature selection uses later versions of an application, evaluation on earlier pairs can accidentally reveal the answer. Splitting by application family, developer, market, and time is often more informative than a random pair split. The same principle applies to AI-era artifacts, where near-duplicate generated outputs from one prompt or agent trajectory can cross train and test boundaries.

Comparison also requires a common unit. Some studies count application pairs, others count applications, clusters, methods, or detected regions. Precision among returned pairs cannot be compared directly with family classification accuracy or top-k retrieval recall. A report should state the candidate population, class balance, threshold-selection set, unresolved labels, and review effort. The framework of Huang et al. and the later RePack critique both motivate this discipline [28, 42].

4.7 What artifact analysis establishes

Android artifact analysis is indispensable because process provenance is frequently absent, incomplete, or controlled by a party under investigation. The literature provides mature techniques for retrieving related candidates, localizing shared regions, resisting selected transformations, and analyzing behavior. Its strongest general contribution is not a universal repackaging verdict. It is a set of observations that can support composition, resemblance, and, with an external directional oracle, derivation.

The principal unresolved issue is evidence binding. A detector output should identify the exact artifact digest, compared region, ignored components, transformation assumptions, threshold, and candidate universe. If direction or authorization is asserted, the report should name the additional source: time, source history, signing identity, watermark

deployment, attestation, or policy. This reporting pattern makes a detector useful in a larger assurance case without asking it to prove what it does not observe.

5 Third-Party Libraries and Component Lineage

Third-party libraries complicate every Android lineage claim. They can dominate an application's code, create similarity among unrelated products, carry vulnerabilities across many packages, and change independently of first-party logic. Removing all library code is not a safe default: the piggybacking study locates injected hooks inside popular library code [45]. Filtering common-code noise and checking component integrity are different tasks. The evidence problem is therefore not simply "filter libraries." It is to identify component boundaries, names, versions, transformations, and use while preserving uncertainty.

The systematic review by Zhan et al. organizes 74 Android-library studies and distinguishes detection, security analysis, maintenance and privilege separation [89]. Its August 2021 author manuscript also reviews permission and storage isolation (Section 4.3): component authorization is not absent from this predecessor, but differs from release authority. Detection itself contains several relations: module separation, family attribution, version selection and vulnerability mapping. Each stage can fail independently.

Zeng et al.'s April 2024 v1 survey extends the discussion across Android, C/C++ and iOS, including usage, updates, risk, patching, dependency management and licensing [86]. Its hardening section also discusses isolation and active learning with regeneration. These subjects are therefore not absent from prior surveys. Section 5.1 explicitly avoids a performance ranking because inconsistent datasets and evaluation contracts prevent one; its technical comparison is not a native common-workload evaluation. The distinction pursued here is between the evidence needed for component attribution, directed derivation, authenticated process and release authority, rather than first coverage of maintenance or security.

5.1 Module separation

An APK does not necessarily preserve the source project's module boundaries. Build tools merge bytecode, shrink unused classes, rewrite names, desugar language features, and package transitive dependencies. A library may be partially included, several libraries may share utility classes, and generated adapters may connect first-party and third-party code. Package names therefore provide useful hints but not a reliable boundary under obfuscation.

LibRadar and LibD illustrate library-detection representations [46, 48]; LibScout, LibID and ORLIS illustrate profile and attribution choices [6, 77, 91]. Together they use different combinations of package structure, normalized method features, class dependencies, call graphs and databases. The design choice affects both recall and the object being reported. A detector that returns individual library classes can remain robust when dead-code elimination removes part of a component, but the result may not correspond to a complete upstream release. A detector that requires a coherent module can provide stronger attribution when it succeeds but may lose partial embeddings.

LibD's boundary recovery combines package inclusion, inheritance and filtered call relations; its opcode-only fingerprints discard operands [46]. Sensitivity to syntactic mutations therefore does not mean sensitivity to every behavior-changing edit. Its 1,000-app comparison uses domain resolution and search as a library-name oracle, not authenticated component boundaries. The reported false-positive percentage counts errors among returned names, not false alarms over a known non-library population. Name recovery, feature-group discovery and exact-version identification must remain separate evaluation units.

Module separation matters for repackaging because shared-library code can produce false similarity. Yet removing candidate libraries before matching can erase relevant evidence when the transformation occurs inside a library or when injected code is placed under a familiar namespace. A sound workflow retains both views: first-party resemblance with common components discounted, and component-aware resemblance that reports which libraries and versions account for the match.

5.2 Family and version attribution

Family attribution maps an in-app region to an upstream library identity. Version attribution is harder because adjacent releases can differ by few methods, while shrinking can remove precisely the distinguishing code. LibScout focuses on reliable library detection and security applications, OSSPolice combines library identification with license and known-risk analysis, and ATVHunter targets reliable version detection for vulnerability identification [6, 17, 87]. LibID and ORLIS explicitly address obfuscation resilience [77, 91].

A version result is conditional on the reference database. LibScout reports unique profiles for 1,225 of 2,065 indexed versions (59.3%); code-only changes can leave consecutive releases with the same profile, yielding a version set rather than a unique answer (Section 6.1) [6]. The detector can select only among indexed releases and may face forks, republished binaries, shaded dependencies, or vendor patches. Reporting a single exact version without alternatives can overstate the evidence. A better output is a candidate set with distinguishing and missing features, plus an "unknown or modified" state when no indexed version explains the observed region.

ATVHunter separates library recall from version recall under a ground truth that excludes apps without complete reference versions [87]. Identical Java across releases leaves multiple version candidates; in a separate shrinking experiment its 75.57% detection rate is below LibPecker's 79.28%. The 66.24-second app-detection result uses the shared 6,819-version comparison database and excludes its construction. A separate full reference database occupies 300 GB after months of collection and feature generation. These are distinct workloads and cost stages.

LibID distinguishes exact-version precision from a candidate set containing the answer [91]. Synthetic tuning does not transfer unchanged to F-Droid apps: LibID-A reports 45% recall with shrinking and class repackaging. Its 38.36-second profiling and 11.25-second matching stages are separate; matching alone is not per-app cost.

Transformation assumptions remain system-specific. ORLIS's class-level matching assumes preserved dependencies, prototypes, and hierarchy rather than arbitrary class splitting/merging or call changes [77]. LibPecker's selected recall oracle is derived from package-name containment, and its matching assumes preserved package hierarchy [92]. Thus, recall within that positive universe is not unbiased recall over all obfuscated libraries. Java-bytecode signatures also do not automatically transfer to native code or ahead-of-time compiled output.

LibPecker's name-selected positive pairs and manually checked returned pairs support different recall and precision claims [92]. Some apparent misses lack a database match despite name agreement, and some correct LibScout family labels have incorrect class correspondences. The experiments cannot form one confusion matrix; family identity does not validate the reported region.

ORLIS can lose short-class signatures before matching and cannot recover candidates discarded by its first filter [77]. Its F-Droid class metrics and reacquired closed-source package totals use different units and not demonstrated byte-identical app versions. Although ORLIS improves the displayed app-analysis stage, preprocessing makes its campaign total 6,763 versus LibDetect's 6,321 CPU minutes. Database-size figures omit JAR repositories; digest timings omit feature construction. The app-stage gain is not a total-lifecycle or storage advantage.

Table 6. Component-evidence tasks and representative systems.

Task and examples	Principal evidence	Ceiling	Important uncertainty
Library-region discovery [46, 48, 77]	Package structure, class relations, calls, normalized code	L1	Partial inclusion, merged modules, generated bridges
Family identification [6, 91, 92]	Signatures compared with an indexed corpus	L1	Forks, unknown libraries, signature collisions, database coverage
Version attribution [17, 87]	Version-discriminating code or graph features	L1	Shrinking, patches, republishing, indistinguishable adjacent releases
Obfuscation-resilient attribution [77, 91]	Normalized classes, dependencies and indexed profiles	L1	Preserved relations, partial code and reference coverage
Library-update studies [15, 64]	Detected or declared library versions across app/source histories	L1	Release order alone does not establish derivation; component versions may remain unresolved
Vulnerability-source studies [78]	Attribution to app or library code with static security findings	L1; L6 for modeled findings	Component presence, exposure, and exploitability need separate evidence
De-duplication and updating [40]	Library separation, compatibility, storage, and update effects	L1; L6 for evaluated behavior	Tested updates do not establish ancestry or behavior in other configurations
Potentially malicious libraries [93]	Library attribution plus security or behavior analysis	L1+L6	Attribution and maliciousness have separate oracles

LibRadar's sub-ten-millisecond lookup excludes loading and a bootstrap exceeding twenty days on ten servers [48]. Incremental updates are proposed, not measured. No observed combined fingerprint collision is not an injectivity proof, and compact fingerprints exclude the full metadata database.

5.3 Obfuscation and transformation resilience

The TPL review distinguishes resilience by feature and stage [89]. Excluding names can suppress identifier-renaming effects; package flattening threatens namespace-based module construction; dead-code elimination removes boundaries and version-discriminating features; control-flow changes affect graphs; and string encryption removes constants. Its Table 9 classifies reported tool capabilities rather than supplying a newly executed common oracle. These conditional comparisons do not establish universal impossibility for every detector or transformation.

The relevant conditional is: given transformation family M, reference corpus D, and inclusion fraction f, can the system identify family or version? "Are We There Yet?" compares tools with different feature granularities and reference databases [88]. Its ProGuard setting disables shrinking, unlike LibID's original F-Droid experiment [91]; online time includes app profiling but not reference-library preparation. Neither recall nor cost can be ranked across those contracts, and reported implementation repairs were not independently reproduced here.

Generated code adds a related transformation. Code generators can copy templates, synthesize adapters, inline dependencies, or produce semantically similar but structurally novel implementations. A library detector may correctly report no known library even though the generated code implements a familiar component interface. Conversely, an agent may import a package based on a textual suggestion, making the dependency a direct supply-chain decision. TPL evidence should distinguish reused implementation from independently generated compatibility code.

5.4 Updates, vulnerabilities, and reachability

Update and vulnerability studies connect composition to maintenance [15, 64, 78], while AppCommune explores automated updating [40]. The chain remains conditional: identify the version and applicable vulnerability; check that affected code survived shrinking; then assess reachability, environment, permissions and execution. LibID finds apps with a vulnerable OkHttp version that do not invoke the affected method [91]. The origins v3 draft's library-associated fractions count vulnerable apps, not all flaws or exploitations; its repeated class pattern is not thousands of independent flaws, and its reachability heuristic omits reflective/native paths [78]. An SBOM or version match addresses only an early link.

KeepUpdated estimates API-compatible upgrade opportunities among detected used-library occurrences [15]. Retained interfaces are its compatibility criterion; behavior changes and new bugs require separate checks. Its Android auto-update discussion is a proposal; iOS signing and Team-Identifier checks illustrate an integrity model. AppCommune requires developer re-signing and tests a subset of patched APIs and selected vulnerabilities [40]. Its storage comparison includes rewritten apps, shared libraries and a manager. Preparation, startup and recovery are separate costs, while future shared-library updates require their own behavior and authorization checks.

Source-history mining finds rollback and integration costs that complicate updates [64]. Its ratings association is not causal; declared dependencies do not certify APK bytes. The retrieved draft's inconsistent update fractions and pattern denominators cannot support a market-wide estimate or certify final-version text.

This layered view prevents two opposite errors. Treating every detected vulnerable version as exploitable inflates risk; treating a library as harmless because an affected method was not observed can miss dynamically loaded or reflective paths. The evidence report should state whether it establishes presence, reachability, triggerability, or observed behavior.

The potentially malicious-library study combines scanner detections with selected harmful-API matches; its comparison cohort is only *possibly* benign [93]. Static paths are not observed harm, and same-package mutations do not prove copying. Its developer graph merges certificate owner text, not public-key fingerprints, so signer identity remains a heuristic despite sample size.

5.5 License, ownership, and common-code ambiguity

OSSPolice separates compliance alerts from legal conclusions and version presence from vulnerability proof [17]. Dual licensing and sublicensing defeat a simple missing-source-link inference; shared SDKs do not prove copying between app developers. Its version precision conditions on detected OSS with discriminating features (83% of indexed C/C++ and 41% of Java versions), so recovery metadata belongs in the comparison contract.

Common components are alternative explanations for application resemblance [43]. First-party residuals can be more probative, but a hard library filter converts component mistakes into clone mistakes; candidate sets should retain uncertainty. In the early common-library study, 810 of 1,050 ad-library candidates remain unresolved, not negative labels. Its simplified downstream classifier is not a native rerun; removing a small list leaves accuracy unchanged and a loose list performs slightly worse than a strict one [43]. More filtering is not necessarily better, and composition is not ancestry.

5.6 A component-aware lineage record

A component-aware lineage record for an APK should contain: the APK digest; candidate first-party regions; detected library regions; family and version candidate sets; omitted or unknown regions; transformation assumptions; reference-database date; and links from each region to the evidence used for attribution. When vulnerability or license claims are made, the record should add reachability or policy evidence rather than silently promoting a composition result.

This record also helps connect artifact analysis to process provenance. An authenticated build record may declare dependencies, while binary analysis observes what was actually packaged. Agreement strengthens confidence. Disagreement is diagnostically useful: the build record may be incomplete, the binary detector may miss a transformed component, or the final artifact may not correspond to the claimed build. The two evidence sources should be reconciled, not treated as redundant.

6 Provenance, Release Authority, and Supply-Chain Integrity

Artifact analysis begins with the product and reasons backward. Supply-chain systems attempt to preserve evidence while the product is created and distributed. This shift changes what can be established. Instead of inferring a possible history from similarity, a provenance system records principals, steps, materials, and products. The record can make direction and authorization explicit, but only within the policy and collection boundary.

Systematizations describe supply-chain defenses through properties, attack classes and causal locations [29, 36, 55]. The secure-design-properties SoK's artifact/operation/actor model is a direct antecedent, not a gap this survey discovers [55]. Its key-bound integrity and external actor authentication are distinct; Section 6 also limits generalization beyond its cases. An APK lies near the end of repositories, registries, workers, signing services and update clients: trustworthy distribution depends on more than the APK signature.

6.1 Repository metadata and delegated authority

TUF's historical root, targets, release and timestamp roles bind hashes, lengths, expiry and versions for retrieval, not installation or source-to-binary correctness [65]. Freeze resistance depends on client time and expiry metadata; root compromise needs trusted recovery. Diplomat adds community delegation under protected-key and trusted-developer assumptions, with possible offline-policy delays [35]. For Android, source approver, builder, signer and store publisher need distinct scoped roles. An APK signature binds a key, not its holder's current authority; rotation, compromise and outsourced builds require policy. Freshness also matters because correctly signed old releases can be vulnerable or revoked across markets and devices.

6.2 Step attestations and source-to-product binding

in-toto signs relations among materials, steps, products and functionaries, then checks an owner-defined layout [75]. An APK product digest can bind a directed source-to-package path. Android layouts would record checkout, dependencies, generation, build/shrink configuration, packaging, signing and publication; generated outputs also need model/service, context and accepted-output bindings. Layout inspections and threshold rebuilders can check configured properties, not unrestricted benign behavior or complete capture. The 30-incident assessment is layout-specific counterfactual analysis, not replay here; its worst package-relative metadata overhead and compressed-repository total use different denominators.

6.3 Transparency and consistent views

CHAINIAC combines developer approval, rebuilding and witnessed history under bootstrap and threshold assumptions [51]. Its evaluated sets include rebuild hash mismatches, and extrapolated history timings are not total rebuild costs. Honest approval may miss harmful code; offline-root recovery does not ensure timely delivery. Contour logs publication commitments [2], but archival availability, inspection and benign behavior remain separate.

Transparency addresses a failure mode that ordinary signatures do not: an authorized key can sign different artifacts for different observers. Logging releases and attestations enables monitors to detect inconsistent or omitted events, subject to log availability and monitoring. In an Android setting, a transparency record can bind package name, version, signing identity, APK digest, build provenance, and publication time. It does not decide whether two versions are semantically equivalent or whether a changed permission is appropriate; those remain analysis questions.

Legitimate binds recorded user intent to commit content and parent/merge fields through client checks and signed records [1]. Its trusted-key and manual-UI assumptions leave whole-commit omission requiring separate reference-state evidence. Cached and selective-object paths already reduce retrieval; the browser path takes longer than cached or native signing when signature cost is included. The user study reports lower task success with the extension, and sequential speed-up may reflect learning. Intent binding does not certify real-person identity, licensing or benign changes.

Contour's extended v2 separates publication commitments, archive availability and monitor inspection [2]. Strong retention assumes honest archivists. Its historical cost account separates per-proof checking, tree storage, auditor bootstrap and archival retention. The initial commitment reuses existing metadata hashes rather than rereading all binary content. Each stage belongs in a corresponding operational comparison.

Transparency can strengthen historical adjudication. If digest A is logged as a build product before candidate B appears in another market, localized resemblance and authenticated chronology support a possible $A \longrightarrow B$ history. They do not exclude siblings derived from an earlier source, nor does a missing record for B establish copying. Direction requires a link to the transformation or another oracle that distinguishes these histories.

6.4 Reproducible builds and independent checking

Reproducible builds compare independent outputs for the same declared inputs and procedure [37]. DetTrace offers deterministic Linux/x86-64 execution under initial-state and well-behaved-program assumptions [39]; external failures, unsupported cases and timeouts are not successes. Its aggregate 3.49× slowdown hides much larger CPU TensorFlow costs against parallel native execution than against serialized native execution. Neither comparator guarantees arbitrary containers or Android builds. For Android, independent rebuilding can bind declared source and dependencies to an APK digest even when signing is centralized.

Reproducibility is often overstated in two ways. First, equality of independent outputs is evidence that the declared procedure is deterministic and that those executions agree; it does not prove that the declared source is authorized or secure. Second, a failed reproduction is diagnostically ambiguous. The cause may be a malicious hidden input, an undeclared dependency, a timestamp, toolchain variation, environment leakage, or ordinary nondeterminism. The result should therefore be reported with input capture and difference localization.

Reproducibility and artifact similarity occupy adjacent but different levels. A reproducible build provides L0 identity for outputs and supports L4 when inputs and procedure are authenticated. A similarity detector can compare releases

that are intentionally non-identical or generated under different toolchains. A mature pipeline uses reproducibility where exact equality is expected and transformation-aware comparison where it is not.

6.5 CI workflows and build-system attack surface

Continuous-integration workflows are executable policy. They select dependencies, obtain credentials, invoke third-party actions, build artifacts, and publish releases. The GitHub CI study analyzes historical bot-event/workflow observations from 2019–2021 [33]; its sampled configurations and commit-message vulnerability proxies are not present-day vulnerability certificates. A signed attestation emitted by a compromised workflow can faithfully record a malicious process. The trustworthiness of provenance therefore depends on who can modify workflow definitions, which actions are pinned, how secrets are scoped, and whether untrusted contributions can influence privileged steps.

That study already distinguishes a pinned action from its mutable transitive dependencies [33]. Its Appendix C also records a false positive: a referenced version lacked a fixing commit but was not exploitable in the reported workflow context. The 38,315 repositories with potentially vulnerable references are therefore not 38,315 confirmed exploitable executions. Its secret analysis similarly separates candidate strings, possible leaks and eleven confirmed accidental leaks; intentionally public limited-service tokens are a different case. These distinctions connect configuration evidence to execution and policy without discarding the underlying integrity concern.

The mobile build adds further attack surfaces: signing keys, store credentials, Gradle plugins, Android SDK components, native toolchains, and remote build caches. A cross-layer assurance case should treat the build configuration and dependency resolver as evidence-bearing artifacts. Their digests and authorization history are part of the lineage of the APK.

A practical implication is that provenance collection must begin before the final packaging step. Recording only the signer and APK digest misses source substitutions, malicious plugins, generated code, and dependency-resolution attacks. At the same time, collecting every event without a policy produces an uninterpretable log. The record should be designed around explicit claims and verification rules.

6.6 Package ecosystems and transitive risk

Package-manager research demonstrates that repositories and dependency ecosystems are active attack surfaces. Early work examined attacks on package managers themselves [10]. Measurements of interpreted-language repositories identified malicious packages and supply-chain attack patterns [16]. Backstabber's Knife Collection cataloged historical malicious-package incidents [53]. Studies of npm found dense dependency relations, maintainer concentration, and weak links that allow risk to propagate beyond a directly selected package [85, 97].

The early package-manager study separates snapshot consistency, freshness and package verification [10]. Its layered signatures rely on uncompromised keys and safe current metadata, not a claim that every signed package is harmless. Client-compromise counts in its 13-day trace use hypothetical vulnerable versions; its 2–5% layering overhead is not a full lifecycle account. The historical Stork deployment is source-reported, not independently validated here.

These findings motivate Android-facing risk hypotheses, not a measured transfer rate between package ecosystems. A mobile application can depend on Maven artifacts, Gradle plugins, JavaScript packages in a hybrid layer, native binaries, model files, and hosted services. An application-level SBOM that lists only direct libraries can omit build-time plugins or generated artifacts. Conversely, listing every transitive package does not establish that its code reached the final APK. Process records and binary component evidence should be reconciled.

These measurements use different oracles. MalOSS combines indicators and limited dynamic interactions with manual review; some findings are registry-confirmed, others unresolved, while its 776-million download estimate includes benign versions rather than unique victims [16]. Backstabber retrieves 174 of 469 identified packages, with some later-stage content missing; its clusters supply technique-reuse leads rather than actor identities [53]. Small World measures transitive reach and simulates vetting, whereas Weak Links uses direct dependents, downloads and stakeholder judgments [85, 97]. In Weak Links, 44.8% endorse the generic installation-script signal versus 67.45% for a specific shell-command variant; maintainer/workload judgments also vary by signal. Its keyword triage yields 63 benign cases among 74 candidates. Operational risk assessment therefore needs the specific signal, population and validation outcome.

SBOM studies also concern different objects. Bi et al.'s 4,786 project discussions describe issue categories, not independently validated SBOMs or fixes; its retrieved revision has conflicting status/timing summaries [7]. It already addresses secure delivery, ownership and other-artifact links. BOMs Away distinguishes content accuracy from transit integrity and proposes consistency and selective-information checks [69], but its abstract and table disagree on practitioner counts, and its proposed solutions leave interoperability, legacy systems and unavailable dependencies unresolved. Neither study demonstrates packaged inclusion or a deployed assurance service.

The distinction between dependency intent and packaged reality is crucial. A lockfile or resolver log says what the build intended or selected. Binary analysis says what appears in the output. Reachability analysis says what can execute. Runtime observation says what executed under sampled inputs. These are four different evidence layers.

6.7 The source-to-APK evidence chain

Figure 2 places the evidence families along a mobile lifecycle. The links are digest and identity bindings. A gap means that records on the two sides may describe different artifacts or principals.

A minimally useful chain records a source revision, resolved dependency set, build and generation configuration, output APK digest, signing identity, release authorization, and publication event. Artifact analysis can then verify the packaged components and compare the output with prior or disputed artifacts. Behavior analysis evaluates the exact digest. These bindings localize disagreements to a particular artifact, transformation, principal or property check.

7 AI-Mediated Transformations and Mobile Lineage

AI-assisted development is not one transformation. It ranges from completion of a few lines to generation of an application skeleton, selection of dependencies, modification across multiple files, test execution, debugging, and release preparation. The relevant provenance question is therefore not merely whether "AI was used." It is which artifact-changing decisions were delegated, which context influenced them, and how the resulting outputs were bound to the final application.

7.1 From text descriptions to mobile artifacts

Text2App's July 2021 v2 translates descriptions through a Simplified App Representation and predefined MIT App Inventor templates to an APK, not unrestricted native Android synthesis [27]. Its synthetic corpus and volunteer evaluation use different outcomes; exact-SAR and functional judgments have unreconciled 111/112 denominators. Compilation does not certify security, and templates and literal processing belong in the accepted-output history.

AppForge evaluates generated Android projects and agents, including compilation-feedback refinement, not authenticated lineage [59]. Functional success counts compile-plus-all-tests over all tasks; test-pass and crash rates condition

Table 7. Supply-chain mechanisms and their Android lineage contribution.

Mechanism	Direct contribution	Ceiling	Residual question
Signed repository metadata and delegation [35, 65]	Target identity, freshness, version, and policy-scoped publishing roles	L5	Whether the target was correctly or safely built
Step attestations and layouts [75]	Authenticated relation among materials, functionary, step, and product	L4–L5	Completeness of capture and trust in execution
Transparency and collective histories [2, 51]	Auditable publication history and resistance to equivocation	L4/L5 with step/policy evidence	Logging alone proves neither build history nor release authority
Reproducible builds [37, 51]	Independent equality check for declared inputs and process	L0; L4 with authenticated inputs	Authorization and security of declared inputs
Signed Git intent records [1]	Commit-content and declared-intent binding	L4 under client/key assumptions	Manual confirmation and key-to-person identity
Deterministic execution [39]	Repeatable execution under declared initial state	Supports equality checking	Supported platform/program assumptions, missing input capture
CI-workflow analysis [33]	Historical risks in machinery that creates attestations and releases	Supports L4 validity	Hidden services, credential compromise, runtime environment
Package-ecosystem measurement (attack and dependency studies discussed above)	Empirical attack and dependency-risk evidence	L1+L6	Exact transfer to the packaged mobile artifact

on compiled apps, and no joint outcome is supplied to infer crash rates among functional successes. Neither success nor a final diff certifies security or records prompts, retrieval, tool state and accepted revisions.

These systems do not make traditional repackaging evidence obsolete. A generated app can itself be copied, repackaged, or combined with existing code. What changes is the space of legitimate relatedness. Multiple authorized outputs may descend from the same prompt, template, or model-mediated plan without one being copied from another. Direction cannot be inferred from pairwise similarity alone.

7.2 Nondeterminism and reproducibility

Conventional build reproducibility assumes that a declared input set and deterministic procedure should yield the same bytes. Generative systems may sample different outputs for the same prompt, call external tools, or depend on changing model and retrieval services. Exact reproduction can therefore be unavailable even when the process is authorized.

This does not eliminate reproducibility as an objective; it changes what must be reproduced. One target is *record reproducibility*: an independent verifier can confirm the prompt or specification reference, model or service identity, parameters, tool sequence, retrieved materials, and accepted outputs. Another is *decision replay*: a recorded output is retained and subsequent deterministic build steps reproduce the APK. A third is *distributional replication*: repeated generations satisfy a defined property, which is an empirical model claim rather than artifact lineage.

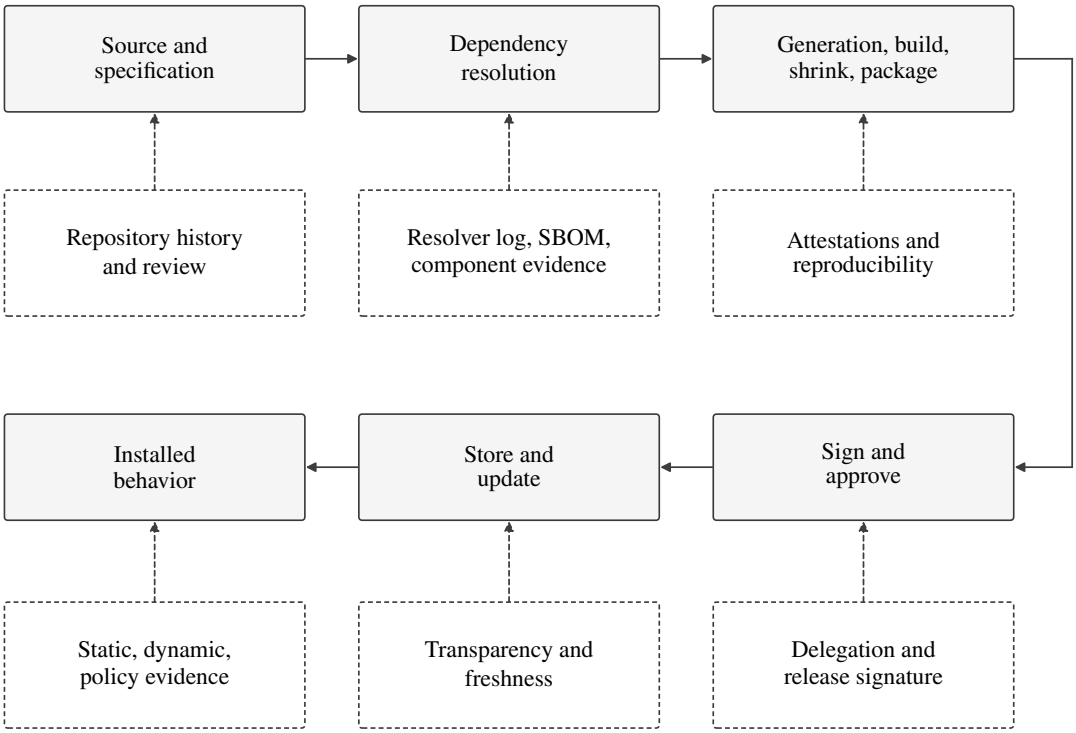


Fig. 2. Android lifecycle and the evidence bound to each stage.

For mobile supply-chain assurance, preserving the accepted generated source or patch is usually more important than reproducing the model's sampling event. The build attestation should treat that accepted output as a material with a digest. If policy requires regeneration, the record must acknowledge that regenerated code may be a sibling rather than an identical ancestor.

7.3 Security of generated code

Studies of code assistants show that generated suggestions can contain security weaknesses and that user interaction affects the result. Pearce et al. evaluated the 2021 Copilot technical preview on targeted security scenarios using CodeQL and manual assessment, without establishing functional correctness [56]. Perry et al.'s 47-participant study used codex-davinci-002 and found less secure assisted results on four of five analyzed tasks, with mixed C-task findings [57]. Sandoval et al. studied 58 student programmers implementing a C shopping-list API and reported a small security impact in that setting [66]. Khoury et al. examined 21 small ChatGPT-3.5 programs against selected attack classes; improvement prompts did not certify general security [31]. Sandoval's metric counts manually annotated CWEs per line of valid code, with a chosen 10% non-inferiority tolerance rather than a zero-harm criterion. The differing tasks, models, participants, and vulnerability oracles preclude a pooled claim that assistance invariably increases insecurity.

For lineage, the important finding is structural: generation provenance does not imply secure behavior. An authorized model-assisted change may introduce a vulnerability; an unauthorized derivative may remove one. Security review must analyze the actual accepted code and built artifact. The provenance record helps identify which model-mediated step introduced a change, but the vulnerability evidence comes from static analysis, testing, review, or formal reasoning.

Within-study oracles also matter. Pearce et al.'s repaired, syntactically valid completions are checked for designated CWEs, not functional correctness or all weaknesses [56]. Perry et al. classify some crash-prone C answers as secure for a task-specific regression; its aggregate $p = 0.056$ and per-task values do not establish uniform adjusted significance [57]. Sandoval et al. find severe-CWE non-inferiority for compiling functions ($p = 0.04$) but not passing functions ($p = 0.06$), testing each submitted function with known-good substitutes for the others [66]. Khoury et al.'s 2026 revision still evaluates historical ChatGPT-3.5 cases, with divergent weakness, correction and execution labels and table/prose disagreements [31]. None is a present-day Android security or general repair rate; their negative and mixed results remain task-specific.

Human approval also needs careful interpretation. Clicking "accept" establishes an interaction event, not necessarily informed review. A policy may require that a maintainer approve generated changes, but the evidential value depends on what was shown, which tests ran, whether warnings were visible, and whether the reviewer had authority for the affected component. Authorization and assurance remain separate.

7.4 Dependencies and package hallucination

Generative systems can recommend dependencies by name. Spracklen et al. measure nonexistent package recommendations in Python and JavaScript generation, using install-command and follow-up-prompt heuristics against PyPI/npm snapshots [68]. Their registry-abuse scenario motivates a dependency risk, but it does not measure an Android attack rate. Transfer to Maven coordinates, Gradle plugins, native packages or download URLs is a hypothesis requiring ecosystem-specific validation; JavaScript dependencies in hybrid apps are closer to the measured setting. The suggestion may enter source or build configuration even when no copied code appears in the model output.

The missing-name fraction counts recommendations, not programs, installs or victims [68]; registry presence is not a trust oracle. The reported mitigation reduces one DeepSeek hallucination rate from 16.14% to 2.66% while HumanEval falls from 51.4 to 25.3. No package was published for the hypothesized abuse. Thus the measured name errors do not establish compromise or a cost-free mitigation.

This risk links AI assistance directly to supply-chain provenance. A robust record should capture the dependency request, resolver result, repository origin, version, integrity digest, and whether the dependency was already approved. A policy can reject unrecognized repositories or require lockfile review. Binary component analysis then checks whether the resolved material appears in the APK. Neither the prompt log nor the binary detector alone provides the full chain.

Package hallucination also motivates a counter-history for library attribution. A newly registered malicious package may intentionally implement an interface or name suggested by a model. The package's code is not necessarily similar to an established library, so clone detection may not flag it. Repository provenance, registration chronology, maintainer identity, and dependency-policy evidence become central.

7.5 Training-data and privacy lineage

CodexLeaks identifies sensitive-output candidates through public-code search and author review [52]. Its 43 validated leakage responses among 2,560 responses are observations under that pipeline, not authenticated membership in an unavailable training corpus. The resulting lineage question differs from ordinary source copying: output association

Table 8. AI-mediated development risks and the evidence needed to reason about them.

Risk or ambiguity	Why final-artifact similarity is insufficient	Complementary evidence	Main claim
Sibling generations (analytical scenario) [27, 59]	Similarity does not reveal whether one output copied another	Specification binding, generation records, accepted-output digests	L3–L4
Insecure generated code [56, 57, 66]	Origin says nothing about security of accepted behavior	Static/dynamic analysis, tests, review evidence, exact artifact binding	L6
Hallucinated dependencies [68]	Malicious package may be novel and not resemble a known library	Resolver provenance, repository identity, lockfile, component verification	L1+L4
Sensitive-output association [52]	Exact training membership and directed ancestry are not authenticated	Model/prompt record, public-source investigation, access-controlled context	Disclosure risk; L3 unresolved
Agentic multi-file edits [59]	A final diff alone need not preserve the intermediate history	Step attestations, tool transcripts, policy checkpoints	L4–L5
Nondeterministic regeneration	Exact output may not be reproducible from a prompt alone	Preserve accepted output; separate record replay from model replication	L0+L4

does not certify the exact training example or a directed repository-copy event. Sensitive material in a final APK is therefore a risk hypothesis requiring artifact-specific investigation, not an outcome measured by this study.

CodexLeaks reports 43 confirmed leaks in 2,560 responses (16.8 per mille); its abstract describes 43 prompts (approximately 8%) [52]. Membership pre-filter precision and recall are imperfect and calibrated on subsequences unlike the direct ten-token responses, for which training membership is unavailable. Public search and adjudication remain necessary; neither a negative search nor historical telemetry certifies absence of memorization or current defenses.

Artifact similarity can still help when a suspected source is known, but absence of a match is weak evidence because generation may transform names and structure. Provenance records can establish which model and prompt produced the accepted output, yet they may not reveal the relevant training example. Policy and legal conclusions require evidence beyond technical resemblance, including license, confidentiality, and permitted-use analysis.

Privacy also affects what provenance should retain. Storing complete prompts, retrieved files, and model outputs may itself expose secrets. A mobile-development provenance design should support selective disclosure: retain cryptographic commitments, access-controlled detailed records, and public metadata sufficient to bind the final artifact without publishing sensitive context. The record needs an evidence-retention policy, not indiscriminate logging.

7.6 Multi-principal and tool-mediated authorship

An agentic workflow can include a human requester, model provider, model instance, retrieval source, code repository, build service, package registry, test system, reviewer, and release signer. These are not equivalent principals. The model service may produce candidate text but lack release authority. A human may approve a patch but not control the signing key. A build service may emit an attestation but not own the source.

Supply-chain layouts are well suited to representing these roles when model-mediated steps are treated as explicit transformations. The layout can require that generated outputs pass tests, static checks, dependency policy, and human review before signing. It can also distinguish advisory output from accepted material. This is more precise than adding one boolean “AI-generated” field to the APK metadata.

The same role separation helps incident response. If a vulnerability is found, investigators can ask whether it originated in a generated patch, retrieved example, dependency recommendation, manual edit, or build-time transformation. The answer guides remediation and model or policy changes. Without step records, the final Git history may collapse several causes into one commit.

7.7 A minimum evidence record for generated transformations

A generated transformation record should identify: the parent source revision; task or specification identifier; model or service identity at the granularity available; generation parameters; tool and retrieval accesses; candidate-output digests; the accepted patch or file digests; tests and analyses run; human or policy approvals; and the downstream build product. Sensitive context can be access controlled, but the public or shared record should still bind the accepted output to the final APK.

This record serves three purposes. It establishes direction from a parent revision to an accepted generated change. It separates generation from approval and release. It gives artifact and behavior analyses exact objects to examine. The record does not prove that the model never used unrecorded training material or that the accepted code is secure. It makes those residual questions explicit rather than hiding them behind a generic provenance claim.

8 Cross-Layer Synthesis: What the Evidence Can Establish

The preceding literatures solve different parts of one problem. Android comparison methods observe the final artifact and recover relations that process records may omit. Component analysis separates first-party logic from dependencies and identifies versions. Supply-chain mechanisms record and authorize production steps. Behavioral analyses evaluate security and policy properties. AI-mediated workflows add transformations whose context may not be represented in source history. The synthesis question is how these observations combine without allowing one layer to stand in for another.

8.1 Six invalid substitutions

Six recurring substitutions expose the missing premise between available evidence and the operationally desired claim:

(1) **Similarity for direction.** A match cannot by itself distinguish $A \to B$, $B \to A$, or a common predecessor. Chronology and ownership can contribute evidence, but a directional oracle must distinguish these histories; an authenticated parent–transformation relation can supply that binding.

(2) **Signer for authorized publisher.** A signature binds an artifact to a key. DroidEagle already rejects certificate presence alone as authenticity evidence [71]. Its trusted-reference fingerprint comparison does not supply the scoped role, validity interval and policy needed for authorization; delegation systems provide policy-scoped publishing authority [35, 65].

(3) **Metadata for benign behavior.** An attestation can bind declared inputs and a build to a digest while recording a vulnerable result. Trusted layout inspections add their configured property checks [75], not unrestricted benign behavior.

(4) **Component presence for exploitability.** A family, version, or SBOM match does not establish that affected code is present, reachable, configured, and exposed; non-detection also does not prove absence after shading, shrinking, patching, or dynamic loading.

(5) **Reproducibility for approval.** Independent builders can reproduce an unauthorized output from unauthorized declared inputs. Reproducibility strengthens equality and source-to-binary binding, not input policy.

(6) **Generation log for complete causal history.** Prompts and responses may omit retrieval, tool state, rejected candidates, manual edits, or downstream transforms. The record must bind the accepted output, parent, and final artifact at the granularity required by the claim.

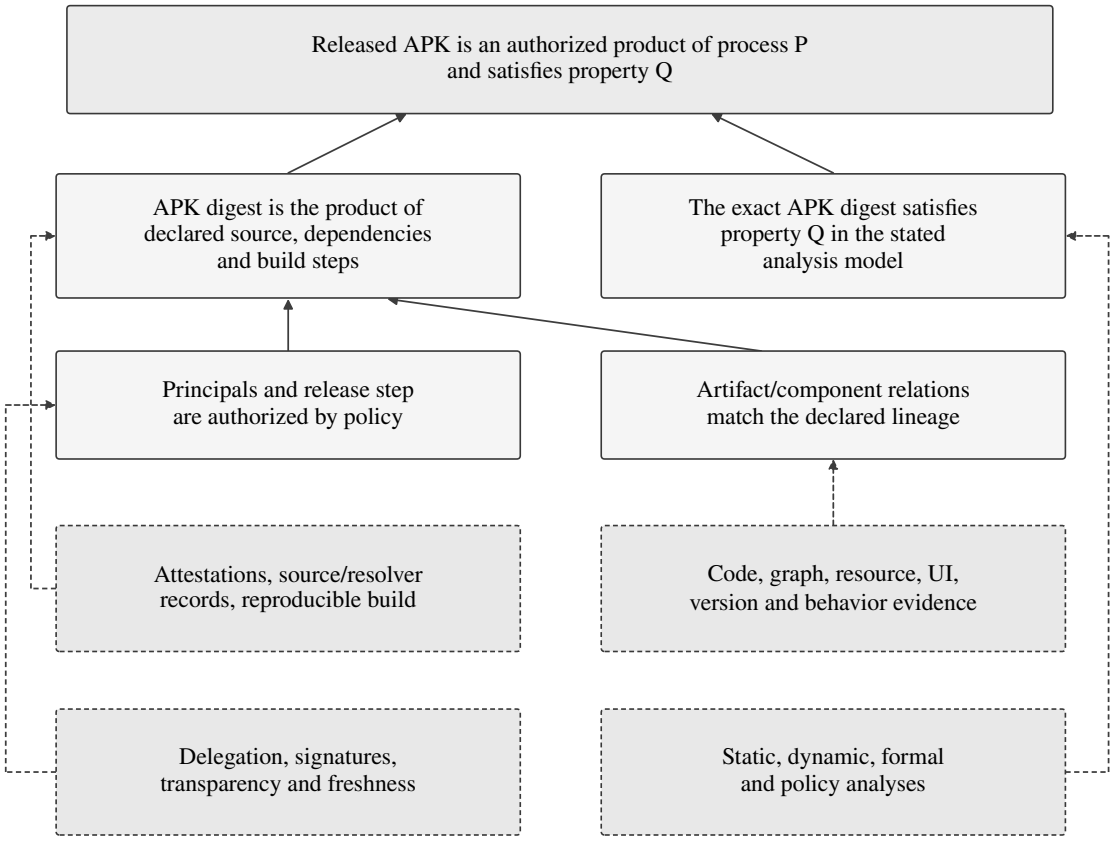


Fig. 3. Cross-layer assurance pattern: solid subclaim arrows and dashed evidence arrows.

These are evaluation errors, not merely wording. Android surveys distinguish privacy and detection [72, 83]; analysis and testing reviews delimit their own coverage [34, 44]. APPraiser and SimiDroid already discuss common builders and multi-level change evidence [30, 41]. Independent generator outputs still require a process record to distinguish them from copying. Direction can be encoded in an oracle, compromised behavior can survive honest-functionary assumptions, and full upstream packages can make versioning easier than partial code. Those are hard negatives, not novelty by absence.

8.2 A cross-layer assurance pattern

Figure 3 presents an Android-focused assurance pattern building on existing artifact/operation/actor properties and supply-chain frameworks [29, 55, 79]. The claim graph identifies the bindings needed to establish that a released Android artifact is an authorized product of a declared process and satisfies a stated property.

The top claim separates authorized production from property Q of the exact APK. Artifact and component analyses cross-check the declared lineage; disagreement requires investigation, not automatic preference for either record.

Table 9. Contradiction patterns and the information needed to resolve or bound them.

Observed disagreement	Likely hidden difference	Required report
Different accuracy or recall	Candidate universe, class balance, threshold, pair construction	Counts by unit, full confusion matrix or retrieval curve, threshold-selection procedure
Different obfuscation robustness	Transformation operators, strength, order, and composition	Transformation matrix and per-operator results
Different library-version results	Database coverage, module granularity, shrinkage, forks	Candidate versions, unknown state, observed/missing discriminators
Attestation and binary scan disagree	Undeclared dependency, detector miss, post-build change, wrong digest	Exact materials, product digest, detector database, packaging step
Signature valid but release disputed	Key compromise, expired role, wrong scope, account takeover	Delegation chain, validity interval, transparency and revocation state
Generated output cannot be reproduced	Nondeterminism, model drift, retrieval change, missing tool state	Accepted-output digest, model/service record, tool transcript, replay boundary

Without trustworthy process evidence, comparison may support a weaker relationship claim. Without behavior analysis, even a fully attested build leaves Q open.

8.3 Binding rules

Cross-layer evidence is useful only when records refer to the same objects. Four binding rules are therefore central.

(1) **Digest binding.** Every attestation, signature, store record, component report, and behavior result should name the artifact digest it covers. Package names and version strings are insufficient because they can be reused or equivocated.
(2) **Predecessor binding.** When artifacts are intentionally non-identical, the record should name the transformation and parent digest. A similarity score alone should not be used as the link when a process record can supply one.
(3) **Principal binding.** Keys, accounts, model services, builders, and reviewers should be mapped to scoped roles and validity intervals. "Signed" is not a complete identity statement.
(4) **Policy binding.** Authorization and behavior conclusions should name the policy version or property definition. A decision can change when the policy changes even if the artifact does not.

The bindings localize disagreements to APK, component database, policy or environment.

8.4 Contradiction patterns

A synthesis preserves incompatible contracts rather than averaging their numbers. RePack distinguishes recall on known-positive pairs from manual checks of returned candidates [42]; AppForge conditions test-pass and crash rates on compiled apps but functional success on all tasks [59]. DetTrace excludes unsupported builds and timeouts, with inconsistent unsupported totals in Table 1 [39]. Transformation sets, output objects (classes, pairs, steps), and threat models also differ. Controlled transformations establish constructed direction but have limited realism; market labels improve realism with uncertain history. ViewDroid accepts shared-builder apps under its criterion whereas ResDroid reports template false alerts [67, 90]. Authenticated histories bind participating projects, not every adversarial artifact. Table 9 states the information required to compare these cases.

Table 10. Minimum reporting checklist for evidence-centered Android lineage evaluation.

Area	Required information	Failure prevented
Claim	Object, relation, claim level, assumptions	Similarity or signature promoted to a stronger claim
Corpus	Source, dates, markets, counts by unit, inclusion and unknowns	Hidden base rates and irreproducible denominators
Oracle	Label construction, direction source, recheck procedure	Circular evaluation and false certainty
Transformations	Operators, parameters, composition, ordinary vs adversarial	Vague robustness claims
Components	Library handling, version database, partial/unknown state	Common-code false positives and overconfident versioning
Provenance	Materials, steps, principals, product digests, layout or policy	Records that cannot be linked to the analyzed APK
Authorization	Role scope, validity, delegation, revocation, transparency	Treating any valid key as rightful publisher
Behavior	Property, tool/model, environment, coverage, limitations	Treating provenance as security assurance
AI mediation	Accepted-output digest, model/service, context boundary, tools, review	Unreplayable or misleading generation history
Operations	Candidate volume, latency, explanation and human review cost	Laboratory metrics that do not scale to deployment

8.5 An evaluation blueprint

A cross-layer evaluation should be organized around falsification. It must name the object and proposition; retain positive, hard-negative, and unknown cases; separate candidate retrieval from verification; include ordered transformation compositions; split by family, developer, market, and time; and recheck a sample of all label states. For generated artifacts, outputs from one prompt family, seed project, or trajectory belong in one partition. For component and vulnerability claims, presence, exact version, reachability, and observed exploit conditions remain separate.

Operational comparison must define a common object and workload before ranking cost: bootstrap/reference construction, per-app extraction, candidate retrieval, pair verification, update insertion, memory, storage, and human adjudication are different stages. Workload, architecture and heap size can reverse rankings; runtime, memory load and warm-up require explicit control [8]. Incremental and multi-stage measurements already exist in the centroid system, AnDarwin and FSquaDRA [11, 14, 94]; their oracle samples and stage boundaries determine the relevant comparison. FSquaDRA's legitimate shared-library clusters also distinguish detector agreement from copy-history validation. Report candidate volume, abstention, explanations and review effort; bind every downstream result to an APK digest or documented predecessor. Table 10 gives the corresponding reporting fields.

8.6 Evidence profiles instead of universal rankings

The synthesis suggests reporting an evidence profile for each system. A profile is a vector over object granularity, claim level, transformation coverage, oracle strength, corpus transfer, and explanation. It avoids a universal ranking that would compare unlike goals. A fast UI detector may be the best first-stage signal for brand impersonation even if it does not establish source derivation. A cryptographic attestation may be decisive for an enrolled build while being

irrelevant to an uncooperative third-party clone. A static analyzer may provide the strongest available privacy evidence while saying nothing about authorship.

Evidence profiles also expose combinations. A deployment can select complementary systems whose failure boundaries differ. For example, component-aware code matching can suppress common-library false positives; a release log can supply chronology, while direction still needs a parent or transformation binding; a delegated signature can provide authorization; and static/dynamic checks can evaluate behavior. The combination should be tested end to end because errors propagate: a wrong component boundary changes the similarity score, a wrong digest breaks provenance binding, and a stale policy changes authorization.

8.7 What the combined literature supports

The framework draws on origin-oracle qualifications in RePack, DroidMOSS and AdRob [21, 42, 96], extended by WuKong, APPraiser and piggybacking [30, 45, 76]; and on the distinction among family, version and vulnerability reach [78, 89, 91]. Steps and delegation close different obligations [35, 65, 75], as do transparency and reproducibility [2, 39, 51]. Assistant-security studies are task-specific [56, 57, 66], while output association and mobile functional tests address other objects [52, 59]. Their key, policy, coverage and oracle assumptions determine the supported proposition. Section 2 records the interpretation boundaries of the corpus.

These observations support a layered case: each contributes its measured proposition, and explicit cross-layer bindings connect those propositions to the analyzed artifact.

9 Research Agenda and Conclusion

The agenda shifts from a universal repackaging score to reviewable claims about artifact relations, production history, authority, and behavior. Detector evaluations should state the conclusion enabled and the premise supplied elsewhere.

9.1 Direction-labeled, transformation-complete benchmarks

Benchmarks should distinguish direct copy, common predecessor, shared library or template, legitimate version, independent generation, and authorized variant. They should record parent/child digests, ordered transformations, first-party/component regions, signing identities, chronology, and label provenance, while separating exact-history controlled cases from realistic field cases and retaining an unknown state.

Android pipelines combine shrinking, desugaring, generation, resource processing, cross-platform compilation, native code, and protection. Because order changes observability, lawful benchmarks should retain per-stage artifacts and report which relation survives each composition.

9.2 Provenance-to-APK and component-to-behavior binding

An Android provenance profile should terminate at the analyzed APK, binding source revision, resolved dependencies, generation/build configuration, builder, signer, publication event, and digest. Sensitive details may remain controlled, but commitments and predecessor relations must be checkable.

Declared composition should be reconciled with the binary. Reports must separate family, candidate version, partial inclusion, modification, reachability, and unknowns; reproducible builds, binary comparison, and transparency should act as independent checks rather than repeated declarations.

9.3 Auditable generated transformations

AI-mediated development needs artifact-level provenance. Each accepted patch or file set should bind a parent revision to a downstream product and record the model/service boundary, material retrieval/tool context, review decision, output digest, and unavailable evidence. Research must determine how to commit to sensitive context, retain sufficient agent history, cluster sibling generations without asserting copying, verify package recommendations, and represent model or tool drift when replay is impossible.

9.4 Open-world, temporal, and uncertainty-aware evaluation

Relatives, developers, libraries, markets, prompt families, and agent trajectories must not leak across random splits. Evaluations should hold out time and entity families, report candidate volume and abstention, keep unknown histories unknown, and distinguish absent, undetected, out-of-scope, and unavailable evidence.

Because keys, roles, packages, policies, and vulnerabilities change, decisions should be reproducible at a stated time and revisable through append-only records and explicit policy versions.

9.5 Decision-centered assurance tooling

Tooling should ingest artifact, component, provenance, signature, transparency, policy, and behavior evidence; check digest/predecessor bindings; and report supported, contradicted, and unresolved subclaims rather than one trust score.

Evaluation should cover retrieval, localization, adjudication, policy action, and appeal. Under a common candidate population, policy, time, and review budget, compare artifact-only, process-only, and bound cross-layer evidence using precision/recall, abstention, reviewer effort, binding success, and localized disagreements. A decision log records the available evidence, policy, abstention or escalation reason, and later corrections, making revisions traceable.

9.6 Stewardship and practice

AndroZoo and RePack show the value of shared data [3, 42]. Resources should preserve label provenance, transformations, licenses, errors, and historical denominators; constrained evaluations can separate public synthetic, redistributable open-source, metadata-only field, and controlled-access strata.

Stores should separate retrieval, lineage, authorization, and behavior review. Developers should retain source-to-APK/dependency bindings, protect signing roles, preserve analyzed artifacts, and record accepted generated changes. Researchers should report the claim ceiling rather than overstate direction or authority.

9.7 Conclusion

Android lineage comprises identity, composition, resemblance, derivation, process, authority, and behavior. Repackaging, TPL, supply-chain, and AI-development research provide complementary evidence for these links.

An assurance case binds source and dependencies to build steps, steps to an APK digest, the digest to an authorized transparent release, and that artifact to component and behavior analyses. Artifact comparison handles absent or disputed records; provenance supplies direction and accountability. This claim-centered structure makes each decision's evidence and assumptions inspectable.

Code availability. Code and accompanying materials are available in the project repository.

References

[1] Hammad Afzali, Santiago Torres-Arias, Reza Curtmola, and Justin Cappos. 2018. Le-git-imate: Towards Verifiable Web-Based Git Repositories. In *ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 469–482. doi:10.1145/3196494.3196523

[2] Mustafa Al-Bassam and Sarah Meiklejohn. 2018. Contour: A Practical System for Binary Transparency. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology (Lecture Notes in Computer Science, Vol. 11025)*. Springer, Cham, Switzerland, 94–110. doi:10.1007/978-3-030-00305-0_8

[3] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *IEEE/ACM International Conference on Mining Software Repositories*. IEEE, Piscataway, NJ, USA, 468–471. doi:10.1145/2901739.2903508

[4] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, and Konrad Rieck. 2014. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 15 pages.

[5] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 259–269. doi:10.1145/2594291.2594299

[6] Michael Backes, Sven Bugiel, and Erik Derr. 2016. Reliable Third-Party Library Detection in Android and Its Security Applications. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 356–367. doi:10.1145/2976749.2978333

[7] Tingting Bi, Boming Xia, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2024. On the Way to SBOMs: Investigating Design Issues and Solutions in Practice. *ACM Transactions on Software Engineering and Methodology* 33, 6, Article 149 (2024), 25 pages. doi:10.1145/3654442

[8] Stephen M. Blackburn, Kathryn S. McKinley, Robin Garner, Chris Hoffmann, Asjad M. Khan, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony L. Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanovic, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. 2008. Wake Up and Smell the Coffee: Evaluation Methodology for the 21st Century. *Commun. ACM* 51, 8 (2008), 83–89. doi:10.1145/1378704.1378723

[9] Pearl Brereton, Barbara A. Kitchenham, David Budgen, Mark Turner, and Mohamed Khalil. 2007. Lessons from Applying the Systematic Literature Review Process within the Software Engineering Domain. *Journal of Systems and Software* 80, 4 (2007), 571–583. doi:10.1016/j.jss.2006.07.009

[10] Justin Cappos, Justin Samuel, Scott Baker, and John H. Hartman. 2008. A Look in the Mirror: Attacks on Package Managers. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 565–574. doi:10.1145/1455770.1455841

[11] Kai Chen, Peng Liu, and Yingjun Zhang. 2014. Achieving Accuracy and Scalability Simultaneously in Detecting Application Clones on Android Markets. In *International Conference on Software Engineering*. IEEE, Piscataway, NJ, USA, 175–186. doi:10.1145/2568225.2568286

[12] Kai Chen, Peng Wang, Yeonjoon Lee, Xiaofeng Wang, Nan Zhang, Heqing Huang, Wei Zou, and Peng Liu. 2015. Finding Unknown Malice in 10 Seconds: Mass Vetting for New Threats at the Google-Play Scale. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 659–674.

[13] Jonathan Crussell, Clint Gibler, and Hao Chen. 2012. Attack of the Clones: Detecting Cloned Applications on Android Markets. In *European Symposium on Research in Computer Security*. Springer, Cham, Switzerland, 37–54. doi:10.1007/978-3-642-33167-1_3

[14] Jonathan Crussell, Clint Gibler, and Hao Chen. 2015. Andarwin: Scalable Detection of Android Application Clones Based on Semantics. *IEEE Transactions on Mobile Computing* 14, 10 (2015), 2007–2019. doi:10.1109/TMC.2014.2381212

[15] Erik Derr, Sven Bugiel, Sascha Fahl, Yasemin Acar, and Michael Backes. 2017. Keep Me Updated: An Empirical Study of Third-Party Library Updatability on Android. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2187–2200. doi:10.1145/3133956.3134059

[16] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 18 pages. doi:10.14722/ndss.2021.23055

[17] Ruian Duan, Ashish Bijlani, Meng Xu, Taesoo Kim, and Wenke Lee. 2017. Identifying Open-Source License Violation and 1-Day Security Risk at Large Scale. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2169–2185. doi:10.1145/3133956.3134048

[18] Ming Fan, Jun Liu, Wei Wang, Haifei Li, Zhenzhou Tian, and Ting Liu. 2017. DAPASA: Detecting Android Piggybacked Apps through Sensitive Subgraph Analysis. *IEEE Transactions on Information Forensics and Security* 12, 8 (2017), 1772–1785. doi:10.1109/TIFS.2017.2687880

[19] Parvez Faruki, Vijay Laxmi, Ammar Bharmal, Manoj Singh Gaur, and Vijay Ganmoor. 2015. AndroSimilar: Robust Signature for Detecting Variants of Android Malware. *Journal of Information Security and Applications* 22 (2015), 66–80. doi:10.1016/j.jisa.2014.10.011

[20] Olga Gadyatskaya, Andra-Lidia Lezza, and Yury Zhauniarovich. 2016. Evaluation of Resource-Based App Repackaging Detection in Android. In *Nordic Conference on Secure IT Systems*. Springer, Cham, Switzerland, 135–151. doi:10.1007/978-3-319-47560-8_9

[21] Clint Gibler, Ryan Stevens, Jonathan Crussell, Hao Chen, Hui Zang, and Heesook Choi. 2013. AdRob: Examining the Landscape and Impact of Android Application Plagiarism. In *International Conference on Mobile Systems, Applications, and Services*. Association for Computing Machinery, New York, NY, USA, 431–444. doi:10.1145/2462456.2464461

[22] Leonid Glanz, Sven Amann, Michael Eichberg, Michael Reif, Ben Hermann, Johannes Lerch, and Mira Mezini. 2017. CodeMatch: Obfuscation Won't Conceal Your Repackaged App. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 638–648. doi:10.1145/3106237.3106305

[23] Betul Gokkaya, Leonardo Aniello, and Basel Halak. 2026. Software Supply Chain: A Taxonomy of Attacks, Mitigations and Risk Assessment Strategies. *Journal of Information Security and Applications* 97 (2026), 104324. doi:10.1016/j.jisa.2025.104324

[24] Hugo Gonzalez, Natalia Stakhanova, and Ali A. Ghorbani. 2014. DroidKin: Lightweight Detection of Android Apps Similarity. In *International Conference on Security and Privacy in Communication Systems*. Springer, Cham, Switzerland, 436–453. doi:10.1007/978-3-319-23829-6_30

[25] Quanlong Guan, Heqing Huang, Weiqi Luo, and Sencun Zhu. 2016. Semantics-Based Repackaging Detection for Mobile Apps. In *International Symposium on Engineering Secure Software and Systems*. Springer, Cham, Switzerland, 89–105. doi:10.1007/978-3-319-30806-7_6

[26] Steve Hanna, Ling Huang, Edward Wu, Saung Li, Charles Chen, and Dawn Song. 2012. Juxtapp: A Scalable System for Detecting Code Reuse among Android Applications. In *Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, Cham, Switzerland, 62–81. doi:10.1007/978-3-642-37300-8_4

[27] Masum Hasan, Kazi Sajeed Mehrab, Wasi Uddin Ahmad, and Rifat Shahriyar. 2021. Text2App: A Framework for Creating Android Apps from Text Descriptions. arXiv:2104.08301. Presented in the NLP4Prog 2021 workshop program; no archival proceedings record was verified.

[28] Heqing Huang, Sencun Zhu, Peng Liu, and Dinghao Wu. 2013. A Framework for Evaluating Mobile App Repackaging Detection Algorithms. In *International Conference on Trust and Trustworthy Computing*. Springer, Cham, Switzerland, 169–186. doi:10.1007/978-3-642-38908-5_13

[29] Eman Abu Ishgair, Juanita Gomez, Marcela S. Melara, Alvaro Cardenas, and Santiago Torres-Arias. 2026. SoK: A Defense-Oriented Evaluation of Software Supply Chain Security. In *2026 IEEE 11th European Symposium on Security and Privacy (EuroS&P)*. IEEE, Piscataway, NJ, USA, 836–859. doi:10.1109/EuroSP68448.2026.00059

[30] Yuta Ishii, Takuya Watanabe, Mitsuaki Akiyama, and Tatsuya Mori. 2017. Appraiser: A Large Scale Analysis of Android Clone Apps. *IEICE Transactions on Information and Systems* E100-D, 8 (2017), 1703–1713. doi:10.1587/transinf.2016ICP0012

[31] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, Piscataway, NJ, USA, 2445–2451. doi:10.1109/SMC53992.2023.10394237

[32] Barbara Kitchenham. 2004. *Procedures for Performing Systematic Reviews*. Technical Report. Keele University.

[33] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of GitHub CI Workflows. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 2747–2763.

[34] Pingfan Kong, Li Li, Jun Gao, Kui Liu, Tegawendé F. Bissyandé, and Jacques Klein. 2019. Automated Testing of Android Apps: A Systematic Literature Review. *IEEE Transactions on Reliability* 68, 1 (2019), 45–66. doi:10.1109/TR.2018.2865733

[35] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. 2016. Diplomat: Using Delegations to Protect Community Repositories. In *USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, Berkeley, CA, USA, 567–581.

[36] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *IEEE Symposium on Security and Privacy*. IEEE, Piscataway, NJ, USA, 1509–1526. doi:10.1109/SP46215.2023.10179304

[37] Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software* 39, 2 (2022), 62–70. doi:10.1109/MS.2021.3073045

[38] Aikaterina Latsiou and Costas Lambrinoudakis. 2026. Cyber Supply Chain Risk Management: From Threats to Treatment. *International Journal of Information Security* 25, 1, Article 40 (2026), 11 pages. doi:10.1007/s10207-025-01207-9

[39] Omar S. Navarro Leija, Kelly Shiptoski, Ryan G. Scott, Baojun Wang, Nicholas Renner, Ryan R. Newton, and Joseph Devietti. 2020. Reproducible Containers. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 167–182. doi:10.1145/3373376.3378519

[40] Bodong Li, Yuanyuan Zhang, Juanru Li, Runhan Feng, and Dawu Gu. 2019. AppCommune: Automated Third-Party Libraries De-Duplicating and Updating for Android Apps. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Piscataway, NJ, USA, 344–354. doi:10.1109/SANER.2019.8668009

[41] Li Li, Tegawendé F. Bissyandé, and Jacques Klein. 2017. SimiDroid: Identifying and Explaining Similarities in Android Apps. In *IEEE International Conference on Trust, Security and Privacy in Computing and Communications*. IEEE, Piscataway, NJ, USA, 136–143. doi:10.1109/Trustcom/BigDataSE/ICESS.2017.230

[42] Li Li, Tegawendé F. Bissyandé, and Jacques Klein. 2021. Rebooting Research on Detecting Repackaged Android Apps: Literature Review and Benchmark. *IEEE Transactions on Software Engineering* 47, 4 (2021), 676–693. doi:10.1109/TSE.2019.2901679

[43] Li Li, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. An Investigation into the Use of Common Libraries in Android Apps. In *IEEE International Conference on Software Analysis, Evolution, and Reengineering*. IEEE, Piscataway, NJ, USA, 403–414. doi:10.1109/SANER.2016.52

[44] Li Li, Tegawendé F. Bissyandé, Mike Papadakis, Siegfried Rasthofer, Alexandre Bartel, Damien Octeau, Jacques Klein, and Yves Le Traon. 2017. Static Analysis of Android Apps: A Systematic Literature Review. *Information and Software Technology* 88 (2017), 67–95. doi:10.1016/j.infsof.2017.04.001

[45] Li Li, Daoyuan Li, Tegawende F. Bissyande, Jacques Klein, Yves Le Traon, David Lo, and Lorenzo Cavallaro. 2017. Understanding Android App Piggybacking: A Systematic Study of Malicious Code Grafting. *IEEE Transactions on Information Forensics and Security* 12, 6 (2017), 1269–1284. doi:10.1109/TIFS.2017.2656460

[46] Menghao Li, Wei Wang, Pei Wang, Shuai Wang, Dinghao Wu, Jian Liu, Rui Xue, and Wei Huo. 2017. LibD: Scalable and Precise Third-Party Library Detection in Android Markets. In *International Conference on Software Engineering*. IEEE, Piscataway, NJ, USA, 335–346. doi:10.1109/ICSE.2017.38

[47] Mario Linares-Vasquez, Andrew Holtzhauer, and Denys Poshyvanyk. 2016. On Automatically Detecting Similar Android Apps. In *IEEE International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 1–10. doi:10.1109/ICPC.2016.7503721

[48] Ziang Ma, Haoyu Wang, Yao Guo, and Xiangqun Chen. 2016. LibRadar: Fast and Accurate Detection of Third-Party Libraries in Android Apps. In *International Conference on Software Engineering Companion*. IEEE, Piscataway, NJ, USA, 653–656. doi:10.1145/2889160.2889178

[49] Enrico Mariconti, Lucky Onwuzurike, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2017. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models. In *Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 15 pages.

[50] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2353–2367. doi:10.1145/3548606.3560596

[51] Kirill Nikitin, Eleftherios Kokoris-Kogias, Philipp Jovanovic, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Justin Cappos, and Bryan Ford. 2017. CHAINIAC: Proactive Software-Update Transparency via Collectively Signed Skipchains and Verified Builds. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 17 pages.

[52] Liang Niu, Shujaat Mirza, Zayd Maradni, and Christina Pöpper. 2023. CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 2133–2150.

[53] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, Cham, Switzerland, 23–43. doi:10.1007/978-3-030-52683-2_2

[54] Marc Ohm and Charlene Stuke. 2023. SoK: Practical Detection of Software Supply Chain Attacks. In *International Conference on Availability, Reliability and Security*. Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/3600160.3600162

[55] Chinenye Okafor, Taylor R. Schorlemmer, Santiago Torres-Arias, and James C. Davis. 2022. SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Association for Computing Machinery, New York, NY, USA, 15–24. doi:10.1145/3560835.3564556

[56] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *IEEE Symposium on Security and Privacy*. IEEE, Piscataway, NJ, USA, 754–768. doi:10.1109/SP46214.2022.9833571

[57] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157

[58] Rahul Potharaju, Andrew Newell, Cristina Nita-Rotaru, and Xiangyu Zhang. 2012. Plagiarizing Smartphone Applications: Attack Strategies and Defense Techniques. In *International Symposium on Engineering Secure Software and Systems*. Springer, Cham, Switzerland, 106–120. doi:10.1007/978-3-642-28166-2_11

[59] Dezhi Ran, Yuan Cao, Mengzhou Wu, Simin Chen, Yuzhe Guo, Jun Ren, Zihe Song, Hao Yu, Jialei Wei, Linyi Li, Wei Yang, Baishakhi Ray, and Tao Xie. 2025. AppForge: From Assistant to Independent Developer—Are GPTs Ready for Software Development? arXiv:2510.07740.

[60] Vaibhav Rastogi, Yan Chen, and Xuxian Jiang. 2013. DroidChameleon: Evaluating Android Anti-Malware against Transformation Attacks. In *ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 329–334. doi:10.1145/2484313.2484355

[61] Beatriz M. Reichert and Rafael R. Obelheiro. 2024. Software Supply Chain Security: A Systematic Literature Review. *International Journal of Computers and Applications* 46, 10 (2024), 853–867. doi:10.1080/1206212X.2024.2390978

[62] Chuangang Ren, Kai Chen, and Peng Liu. 2014. DroidMarking: Resilient Software Watermarking for Impeding Android Application Repackaging. In *IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, 635–646. doi:10.1145/2642937.2642977

[63] Israel J. Mojica Ruiz, Meiyappan Nagappan, Bram Adams, and Ahmed E. Hassan. 2012. Understanding Reuse in the Android Market. In *IEEE International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 113–122. doi:10.1109/ICPC.2012.6240477

[64] Pasquale Salza, Fabio Palomba, Dario Di Nucci, Cosmo D'Uva, Andrea De Lucia, and Filomena Ferrucci. 2018. Do Developers Update Third-Party Libraries in Mobile Apps?. In *IEEE International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 255–265. doi:10.1145/3196321.3196341

[65] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. 2010. Survivable Key Compromise in Software Update Systems. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 61–72. doi:10.1145/1866307.1866315

[66] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 2205–2222.

[67] Yuru Shao, Xiapu Luo, Chenxiong Qian, Pengfei Zhu, and Lei Zhang. 2014. Towards a Scalable Resource-Driven Approach for Detecting Repackaged Android Applications. In *Annual Computer Security Applications Conference*. Association for Computing Machinery, New York, NY, USA, 56–65. doi:10.1145/2664243.2664275

[68] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2025. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 3687–3706.

[69] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M. German, and Denys Poshyvanyk. 2024. BOMs Away! Inside the Minds of Stakeholders: A Comprehensive Study of Bills of Materials for Software Systems. In *2024 IEEE/ACM 46th International Conference*

on Software Engineering (ICSE). Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3597503.3623347

[70] Sufatrio, Darell J. J. Tan, Tong-Wei Chua, and Vrizlynn L. L. Thing. 2015. Securing Android: A Survey, Taxonomy, and Challenges. *Comput. Surveys* 47, 4 (2015), 58:1–58:45. doi:10.1145/2733306

[71] Mingshen Sun, Mengmeng Li, and John C. S. Lui. 2015. DroidEagle: Seamless Detection of Visually Similar Android Apps. In *ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/2766498.2766508

[72] Kimberly Tam, Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, and Lorenzo Cavallaro. 2017. The Evolution of Android Malware and Android Analysis Techniques. *Comput. Surveys* 49, 4 (2017), 76:1–76:41. doi:10.1145/3017427

[73] Wei Tang, Ping Luo, Jialiang Fu, and Dan Zhang. 2020. LibDX: A Cross-Platform and Accurate System to Detect Third-Party Libraries in Binary Code. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Piscataway, NJ, USA, 104–115. doi:10.1109/SANER48275.2020.9054845

[74] Ke Tian, Danfeng Yao, Barbara G. Ryder, Gang Tan, and Guojun Peng. 2020. Detection of Repackaged Android Malware with Code-Heterogeneity Features. *IEEE Transactions on Dependable and Secure Computing* 17, 1 (2020), 64–77. doi:10.1109/TDSC.2017.2745575

[75] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 1393–1410.

[76] Haoyu Wang, Yao Guo, Ziang Ma, and Xiangqun Chen. 2015. WuKong: A Scalable and Accurate Two-Phase Approach to Android App Clone Detection. In *ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 71–82. doi:10.1145/2771783.2771795

[77] Yan Wang, Haowei Wu, Hailong Zhang, and Atanas Rountev. 2018. ORLIS: Obfuscation-Resilient Library Detection for Android. In *IEEE/ACM International Conference on Mobile Software Engineering and Systems*. Association for Computing Machinery, New York, NY, USA, 13–23. doi:10.1145/3197231.3197248

[78] Takuya Watanabe, Mitsuaki Akiyama, Fumihiro Kanei, Eitaro Shioji, Yohei Takata, Bo Sun, Yuta Ishii, Toshiki Shibahara, Takeshi Yagi, and Tatsuya Mori. 2017. Understanding the Origins of Mobile App Vulnerabilities: A Large-Scale Measurement Study of Free and Paid Apps. In *IEEE/ACM International Conference on Mining Software Repositories*. IEEE, Piscataway, NJ, USA, 14–24. doi:10.1109/MSR.2017.23

[79] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, Michel Cukier, Christian Kästner, Alexandros Kapravelos, Dominik Wermke, and William Enck. 2025. Research Directions in Software Supply Chain Security. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 146:1–146:38. doi:10.1145/3714464

[80] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/2601248.2601268

[81] Yiming Wu, Zhuowu Liu, Yanjie Lin, Binbin Zhao, Chunyi Zhou, Tiejun Wu, and Zhen Hong. 2026. Android App Repackaging Detection: A Comprehensive Survey. *Cyber Security and Applications* 4 (2026), 100124. doi:10.1016/j.csa.2026.100124

[82] Jian Xu and Qianting Yuan. 2022. LibRoad: Rapid, Online, and Accurate Detection of TPLs on Android. *IEEE Transactions on Mobile Computing* 21, 1 (2022), 167–180. doi:10.1109/TMC.2020.3003336

[83] Meng Xu, Chengyu Song, Yang Ji, Ming-Wei Shih, Kangjie Lu, Cong Zheng, Ruian Duan, Yeongjin Jang, Byoungyoung Lee, Chenxiong Qian, Sangho Lee, and Taesoo Kim. 2016. Toward Engineering a Secure Android Ecosystem: A Survey of Existing Techniques. *Comput. Surveys* 49, 2 (2016), 38:1–38:47. doi:10.1145/2963145

[84] Shengtao Yue, Weizan Feng, Jun Ma, Yanyan Jiang, Xianping Tao, Chang Xu, and Jian Lu. 2017. RepDroid: An Automated Tool for Android Application Repackaging Detection. In *IEEE/ACM International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 132–142. doi:10.1109/ICPC.2017.16

[85] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What Are Weak Links in the npm Supply Chain?. In *International Conference on Software Engineering: Software Engineering in Practice*. IEEE, Piscataway, NJ, USA, 331–340. doi:10.1145/3510457.3513044

[86] Jia Zeng, Dan Han, Yaling Zhu, Yangzhong Wang, and Fangchen Weng. 2024. A Survey of Third-Party Library Security Research in Application Software. arXiv:2404.17955.

[87] Xian Zhan, Lingling Fan, Sen Chen, Feng Wu, Tianming Liu, Xiapu Luo, and Yang Liu. 2021. ATVHunter: Reliable Version Detection of Third-Party Libraries for Vulnerability Identification in Android Applications. In *International Conference on Software Engineering*. IEEE, Piscataway, NJ, USA, 1695–1707. doi:10.1109/ICSE43902.2021.00150

[88] Xian Zhan, Lingling Fan, Tianming Liu, Sen Chen, Li Li, Haoyu Wang, Yifei Xu, Xiapu Luo, and Yang Liu. 2020. Automated Third-Party Library Detection for Android Applications: Are We There Yet?. In *IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 919–930. doi:10.1145/3324884.3416582

[89] Xian Zhan, Tianming Liu, Lingling Fan, Li Li, Sen Chen, Xiapu Luo, and Yang Liu. 2022. Research on Third-Party Libraries in Android Apps: A Taxonomy and Systematic Literature Review. *IEEE Transactions on Software Engineering* 48, 10 (2022), 4181–4213. doi:10.1109/TSE.2021.3114381

[90] Fangfang Zhang, Heqing Huang, Sencun Zhu, Dinghao Wu, and Peng Liu. 2014. ViewDroid: Towards Obfuscation-Resilient Mobile Application Repackaging Detection. In *ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, USA, 25–36. doi:10.1145/2627393.2627395

[91] Jiexin Zhang, Alastair R. Beresford, and Stephan A. Kollmann. 2019. LibID: Reliable Identification of Obfuscated Third-Party Android Libraries. In *ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 55–65. doi:10.1145/3293882.3330563

[92] Yuan Zhang, Jiarun Dai, Xiaohan Zhang, Sirong Huang, Zhemin Yang, Min Yang, and Hao Chen. 2018. Detecting Third-Party Libraries in Android Applications with High Precision and Recall. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Piscataway, NJ, USA, 141–152. doi:10.1109/SANER.2018.8330204

[93] Zicheng Zhang, Wenrui Diao, Chengyu Hu, Shanqing Guo, Chaoshun Zuo, and Li Li. 2020. An Empirical Study of Potentially Malicious Third-Party Libraries in Android Apps. In *ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, USA, 144–154. doi:10.1145/3395351.3399346

[94] Yury Zhauniarovich, Olga Gadyatskaya, Bruno Crispo, Francesco La Spina, and Ermanno Moser. 2014. FSquaDRA: Fast Detection of Repackaged Applications. In *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, Cham, Switzerland, 130–145. doi:10.1007/978-3-662-43936-4_9

[95] Wu Zhou, Xinwen Zhang, and Xuxian Jiang. 2013. AppInk: Watermarking Android Apps for Repackaging Deterrence. In *ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/2484313.2484315

[96] Wu Zhou, Yajin Zhou, Xuxian Jiang, and Peng Ning. 2012. Detecting Repackaged Smartphone Applications in Third-Party Android Marketplaces. In *ACM Conference on Data and Application Security and Privacy*. Association for Computing Machinery, New York, NY, USA, 317–326. doi:10.1145/2133601.2133640

[97] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *USENIX Security Symposium*. USENIX, Berkeley, CA, 995–1010.